

Going Faster: Privacy-Preserving Shortest Paths from Start to End

Vincent Ehrmanntraut

RWTH Aachen University

Aachen, Germany

ehrmanntraut@itsec.rwth-aachen.de

Ulrike Meyer

RWTH Aachen University

Aachen, Germany

meyer@itsec.rwth-aachen.de

Abstract

Finding shortest paths in graphs is one of the fundamental combinatorial optimization problems with numerous applications. Privacy-constraints in these applications have lead to an extensive line of research on the so-called privacy-preserving (length of) shortest path problem. A Secure Multi-Party Computation (SMPC) protocol that solves this problem computes the lengths of shortest paths on a secret graph in a distributed fashion while ensuring that the graph remains secret. While many such protocols have been proposed in the past, they only compute the length of the shortest paths and not the paths themselves.

In this paper, we address this shortcoming but also propose a novel adaptable protocol design that chooses between different sub-protocols for the building blocks it uses depending on input size, the network delay and bandwidth, and the security model the protocol operates in. Our protocol thus finds the run-time optimal combination of sub-protocols for a given environment.

We compare the resulting adaptive protocol to a less flexible baseline protocol in an extensive evaluation using the MP-SPDZ framework, spanning multiple network and security settings. Compared to the baseline protocol, the adaptivity of our optimized path construction leads to speedups from 4% to 32% in the overall time to find the shortest path.

Further, we find and fix a small leakage in two prior (length of) shortest path protocols, and report multiple observations that can be used to improve the practical runtime of other SMPC protocols.

Keywords

secure multi-party computation; shortest path

1 Introduction

The shortest path problem consists of finding a path of minimal length between nodes in a graph $G = (V, E, l)$ with edge lengths l . It is one of the fundamental combinatorial optimization problems [16] with numerous applications, ranging from simple navigation to complex problems such as the min-cost flow problem that iteratively find solutions by searching shortest paths in changing graphs [14]. Many of these applications have privacy-constraints, such as the example application discussed in Appendix A.

Recently, such applications have been a focus of research [2, 5, 43] using Secure Multi-Party Computation (SMPC), powerful cryptographic primitives that allow computations on private data, to build protocols where multiple parties are able to perform shortest path computations in a distributed way without any party having knowledge of the graph. Typically, these protocols assume that the graph already exists in a secret-shared form, i. e., that a different,

application-specific, protocol has already built and shared the input graph based on some application-specific private data of the parties. The protocols then perform their computations on this secret graph, and produce secret-shared outputs, which then can be used by a following application-specific protocol. This allows investigating the privacy-preserving shortest path problem independently of its applications.

Using this principle, numerous protocols have been proposed to solve both the privacy-preserving all-pairs shortest path problem [5], which finds the distances between each pair of nodes, and the single-source shortest path problem [2, 3, 5, 7, 35, 40, 43], which finds the distances between a single node and all other nodes. However, these protocols only compute the lengths of the shortest paths, and do not produce the corresponding paths themselves.

In this paper, we study the single-pair shortest path problem, where a start node and a target node are given. Compared to prior work, our protocol returns a shortest path and not just its length. In addition, we reckon that SMPC protocols must be tailored towards the application's computation environment in terms of input size, network delay and bandwidth as well as the desired security model, as each environment requires different trade-offs between the number of communication rounds, the total data sent, and computation.

We therefore dissect the task of finding a shortest path into two distinct subtasks. The first is determining the existence of a shortest path as well as determining the successor node for each node on the shortest path in a privacy-preserving manner. The second subtask is constructing the actual path from this secret shared information.

To construct efficient sub-protocols to solve the first task, we modify the most efficient (length of) shortest paths protocols previously proposed [5] by (1) fixing a small leakage found in one of them and (2) enhancing them to compute the successor nodes while determining the length of the shortest path. While this enhancement is straightforward in principle, realizing this approach as an efficient SMPC protocol has proven itself to be surprisingly challenging. For example, producing successor nodes instead of the usual predecessor nodes requires graph-theoretic trickery.

To solve the second task, we propose to combine a building block for demultiplexing to construct a successor matrix and a building block to follow the successors in a successor matrix for the final path extraction. For each of these building blocks we identify or newly propose multiple sub-protocols, each coming with its own trade-off between communication rounds, amount of data sent and local computation required.

Our overall shortest path protocol is then able to combine the optimal sub-protocols for a given computation environment. This new paradigm in protocol design departs from prior work that mainly focuses on optimizing communication rounds.

We confirm the benefit of this new approach in an extensive evaluation with multiple network settings and different security



This work is licensed under a Creative Commons Attribution 4.0 International License.

models. We compare our adaptive protocol to a baseline protocol that combines our modified versions of the length of shortest path protocols of [2, 5] with a path reconstruction proposed in [12] and show that our protocol reduces the overall single-pair shortest path runtime by up to 44% on small graphs. On larger graphs, the successor calculation dominates the overall runtime, and optimizations in the path construction have less impact. Still, on graphs with 1000 nodes, using our optimized path construction leads to a 3.8% reduction in the overall runtime in a fast network setting, and 7.5% in a slow network setting.

To summarize, our contributions are as follows:

- We identify and fix a subtle leakage in two prior shortest path protocols, namely the Dijkstra protocols by Aly and Cleemput [2] and by Anagreh et al. [5].
- We then modify and extend these “length of shortest path” protocols, that only compute distances, to an adaptive protocol that actually produces a shortest path. This includes novel techniques to ensure that the path is well-formed and to hide the number of nodes on the shortest path.
- We also present a technically beautiful but practically slow protocol requiring only $\mathcal{O}(\log^2 n)$ rounds in Appendix B.
- The performance of the adaptive protocol, the baseline protocol, and the round-optimized protocol are extensively evaluated in multiple network and security settings on graphs with up to 1000 nodes. We show that compared to the baseline our optimizations to the path construction result in speedups from 3.8% up to 44% on the overall runtime, which includes the successor calculation.
- Finally, we perform microbenchmarks of our building blocks in the MP-SPDZ framework, leading to a range of surprising results that may also be interesting for other SMPC protocols. For example, we find that in MP-SPDZ less-than comparisons are roughly two times faster than equals comparisons. We also find that the fastest approach to perform a demultiplex, a common building block to convert a numerical value into the corresponding unit vector, not only depends on the network setting and input size, but also on the number of demultiplexes performed in parallel.

2 Preliminaries

This section briefly introduces Secure Multi-Party Computation, along with the Arithmetic Black Box (ABB) in Section 2.1, followed by the notation used to describe this paper’s protocols in Section 2.2. Finally, Section 2.3 provides a quick overview of relevant definitions from graph theory.

2.1 Secure Multi-Party Computation

SMPC allows multiple parties to jointly perform a computation on private inputs without revealing them to other parties. Such computations can be realized using a range of primitives [8, 21, 26, 31, 44, 54] that differ in the supported number of parties, and the provided security guarantees, where semi-honest security assumes that all parties strictly adhere to the protocol, and malicious security allows arbitrary deviations. These primitives provide elementary operations like additions and multiplications that can be assembled into higher-level functionalities, which typically are

realized as circuits consisting of elementary operations [2, 5, 33]. As a consequence, most high-level protocols can be instantiated with a range of SMPC primitives, as long as the primitives provide all necessary elementary operations.

- **INPUT (v):**
Store the integer input v in private memory and return its handle $[v]$. The input can either be public, i. e., be provided by all parties, or private, i. e., be provided by a subset of parties.
- **REVEAL ($[v]$):**
Reveal the value associated with the handle $[v]$ to all parties.
- **LINEARCOMBINATION ($c_1, [v_1], \dots, c_k, [v_k]$):**
Calculate $r = \sum_{i=1}^k c_i \cdot v_k$, store it in the secure memory, and return its new handle $[r]$.
- **MULTIPLY ($[v_1], [v_2]$):**
Calculate $r = v_1 \cdot v_2$, store it in the secure memory, and return its new handle $[r]$.
- **LESTHAN ($[v_1], [v_2]$):**
Store $b = 1$ in the secure memory if $v_1 < v_2$, or $b = 0$ if $v_1 \geq v_2$, and return the handle $[b]$.
- **EQUALS ($[v_1], [v_2]$):**
Store $b = 1$ in the secure memory if $v_1 = v_2$, or $b = 0$ if $v_1 \neq v_2$, and return the handle $[b]$.
- **BITDECOMPOSE ($[v], k$):**
Determine the k least significant bits $b_1, \dots, b_k$ of v , store them in secure memory and return their handles $[b_1], \dots, [b_k]$.
- **GENERATEPERMUTATION (k):**
Generate a uniformly random permutation of $1, \dots, k$, and return its handle $[\sigma]$.
- **SHUFFLE ($[v_1], \dots, [v_k], [\sigma]$) and UNSHUFFLE ($[v_1], \dots, [v_k], [\sigma]$):**
Shuffle the vector $[v_1], \dots, [v_k]$ using the permutation σ / σ^{-1} , and return handles to the shuffled vector.

Figure 1: Commands supported by the ABB.

This relationship is formalized by the Arithmetic Black Box (ABB), an ideal functionality that can securely store private information, as well as perform elementary operations on the private data [22]. When private data is stored in the ABB, all parties obtain a handle (read pointer) of the private data. To perform operations on private data, the parties send commands to the ABB using handles to specify the inputs, and obtain handles of the resulting values. The list of available commands is displayed in Figure 1. Notably, it is a specific variant of the ABB introduced by Anagreh et al. [5] that includes comparisons and shuffling.

The ABB facilitates security proofs significantly: Low-level primitives only have to prove that they implement the ABB securely, and high-level protocols only have to prove that they are secure assuming that the ABB is secure. Concretely, high-level protocols only have to show that revealed values and the control flow, i. e., the issued commands, do not reveal anything about the private inputs [2, 5, 22]. Importantly, data-oblivious protocols that do not reveal any secret values are trivially secure in the ABB model.

In summary, the ABB is a useful abstraction layer to separate high-level protocols from the underlying primitives used to perform elementary operations. However, to understand the design of efficient protocols, a brief discussion of the primitives' performance characteristics is required: We use the current state-of-the-art SMPC primitives [1, 8], that secret-share the private data amongst the parties. Elementary operations are performed on the secret-shares. This allows efficient additions of secret values using local computations only, i. e., without communication between the parties. Multiplications and comparisons require the parties to communicate, leading to a communication overhead composed of the network delay and sending time imposed by the network's bandwidth. The network delay's impact is commonly reduced by performing operations in parallel and bundling their messages into few large messages instead of sending many small messages. This naturally leads to three different complexity metrics: The round complexity reflects the number of communication steps required during the protocol's execution, while the communication complexity indicates how much data is sent in total. Finally, the computation complexity is determined by the total number of computation steps.

2.2 Notation

Instead of describing the protocols in this paper as a sequence of ABB commands, we use Toft's notation [48] in our protocol descriptions: A handle on the secret value x is denoted as $[x]$, publicly known values are written without brackets. $[a] + [b]$ is the addition of secret values a and b . Analogously, $[a] \cdot [b]$ is the multiplication of a and b , and $[a] < [b]$ the comparison of a and b . Further, we use $[c] ? [a] : [b]$ as shorthand notation for $[a] + [c] \cdot ([b] - [a])$, which results in $[a]$ if $c = 0$, and $[b]$ if $c = 1$.

Our protocols also make use of vectors and matrices. Vectors are written with an arrow and are capitalized, $\vec{V}_i$ is the i 'th entry of the vector $\vec{V}$. Matrices also are capitalized and bold, with $M_{i,j}$ being the element in the i 'th row and j 'th column of matrix M . The whole i 'th row is written as $M_{i,*}$, and the j 'th column as $M_{*,j}$. For secret vectors and matrices, the notation is analogous.

When a vector is shuffled using the permutation σ , which has to be picked beforehand, $\sigma(i)$ is the new position of the element at the old position i . A matrix is shuffled by first shuffling each row individually with the same permutation, and then shuffling each column individually, again using the same permutation. This matrix shuffling procedure will later correspond to shuffling the nodes of a graph.

2.3 Graph theory

A graph $G = (V, E)$ with edge lengths l consists of a set of nodes V , a set of edges $E \subseteq V \times V$ connecting some of these nodes, and an edge length function $l : V \times V \mapsto \mathbb{Z} \cup \{\infty\}$ ¹. The edge length function maps edges $u, v \in E$ to their length, and non-edges $u, v \notin E$ to ∞ .

We assume, without loss of generality, that $V = \{1, \dots, n\}$, where $n = |V|$ is the number of nodes. This allows presenting the edge lengths as matrix L with $L_{i,j} = l(i, j)$. Further, this allows encoding nodes as vectors, where a node $v \in V$ is mapped to a vector of length n containing only zeroes except a singular one at position v .

These representations of the nodes and edge lengths are useful to express some graph operations as vector-matrix products.

Finally, a path from $s \in V$ to $t \in V$ is a collection of nodes $p = (v_1, \dots, v_k)$ such that $v_1 = s, v_k = t$, and $v_i, v_{i+1} \in E$ for all $1 \leq i < k$. Its number of steps equals the number of traversed edges, i. e., $k - 1$, and its length is the sum of all edge lengths, i. e., $l(p) = \sum_{i=1}^{k-1} l(v_i, v_{i+1})$. A shortest path from s to t is a path whose length is minimal, i. e., there is no shorter path from s to t ². The problem of finding such paths from a starting node to a target node is called the single-pair shortest path problem. It should be noted that this problem is only well-defined when the graph contains no cycles with a negative total length.

3 Related work

Prior privacy-preserving shortest path protocols are presented in Section 3.1, followed by an overview of novel non-privacy-preserving algorithms in Section 3.2. Finally, Section 3.3 presents a related but orthogonal approach to optimizing SMPC protocols.

3.1 Privacy-preserving shortest paths

Multiple SMPC protocols that solve the privacy-preserving shortest path problem have been proposed. Generally, these can be divided into protocols that operate on sparse graphs, where the number of edges m is in the order of the number of nodes n , and protocols that operate on dense graphs, where the number of edges is close to n^2 .

Protocols on sparse graphs mostly adapt Dijkstra's algorithm using Oblivious Random Access Memories (ORAMs) to hide memory access patterns [5, 35, 40, 43]. The current state-of-the-art in this research direction exploits the fact that each node is visited only once to achieve a protocol requiring $\mathcal{O}(n \log n \log \log n)$ rounds [43]. However, the protocol only computes the length of shortest paths, and does not yield the paths themselves.

On dense graphs, Aly and Van Vyve [3] first introduced a protocol inspired by Dijkstra's algorithm which hides memory access patterns by accessing all possible edges in each iteration. Even though some optimizations reduce the search space in later iterations, this leads to a high overhead.

Aly and Cleemput [2] reduce this overhead using the approach depicted in Protocol 1. It first shuffles the nodes during the initialization of the protocols. This then allows revealing the shuffled index of the closest unvisited node to the source, enabling efficient updates of the neighboring nodes' distances. Anagreh et al. [5] further optimize this approach by using a more efficient method to find the closest unvisited node in Line 8 of Protocol 1. It should be noted that, while the fundamental approach can be realized securely, both protocols suffer from a small leakage. Section 4 further discusses the leakage and its prevention.

Further, these protocols do not return the shortest paths. Instead, the Dijkstra protocol by Anagreh et al. [5] only computes the lengths of shortest paths. Aly and Van Vyve [3] and Aly and Cleemput [2] also include the predecessor nodes. As shown in Section 5, constructing a path from these predecessor nodes is not trivial.

Anagreh et al. [5] also adapt other shortest path algorithms to privacy-preserving protocols, including Floyd-Warshall [30] and min-addition [11] for dense graphs, and Bellman-Ford [10] for

¹ ∞ is a large constant in practice.

²Note that there might be multiple shortest paths of equal length from s to t .

Table 1: Chronologically-ordered comparison of SMPC protocols for the privacy-preserving shortest path and Breadth-First-Search (BFS) problems. Complexities ignore polynomial dependencies on the number of parties and field size.

Protocol	Base Algorithm	Returns Path	Rounds	Communication	Computation
Blanton et al. [12] *	BFS	●	$\mathcal{O}(n^2 + S)^*$	$\mathcal{O}(n^2 + S)^*$	$\mathcal{O}(n^2 + S)^*$
Keller and Scholl [35] Δ	Dijkstra	○	$\mathcal{O}(n \log^3 n + m \log^4 n)$	$\mathcal{O}(n \log^3 n + m \log^4 n)$	$\mathcal{O}(n \log^3 n + m \log^4 n)$
Aly and Van Vye [3]	Dijkstra	●	$\mathcal{O}(n^3)$	$\mathcal{O}(n^5)$	$\mathcal{O}(n^5)$
Liu et al. [40] Δ	Dijkstra	○	$\mathcal{O}((n + m) \log^2 n)$	$\mathcal{O}((n + m) \log^2 n)$	$\mathcal{O}((n + m) \log^2 n)$
Aly and Cleemput [2] $\dagger$	Dijkstra	●	$\mathcal{O}(n^2 + S)^*$	$\mathcal{O}(n^2 \log n + S)^*$	$\mathcal{O}(n^2 \log n + S)^*$
Anagreh et al. [5] $\dagger$	Dijkstra	○	$\mathcal{O}(n \log n + S)^*$	$\mathcal{O}(n^2 + S)^*$	$\mathcal{O}(n^2 + S)^*$
Anagreh et al. [5]	Floyd-Warshall	○	$\mathcal{O}(n)$	$\mathcal{O}(n^3)$	$\mathcal{O}(n^3)$
Anagreh et al. [5]	Min-Addition	○	$\mathcal{O}(\log^2 n)$	$\mathcal{O}(n^3 \log n)$	$\mathcal{O}(n^3 \log n)$
Anagreh et al. [5] Δ	Bellman-Ford	○	$\mathcal{O}(n \log n)$	$\mathcal{O}(nm)$	$\mathcal{O}(nm)$
Anagreh et al. [6] $\ddagger$	Dijkstra	○	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2) / \mathcal{O}(n^3)^\diamond$	$\mathcal{O}(n^3)$
Ostrovsky [43] Δ	Dijkstra	○	$\mathcal{O}(n \log n \log \log n)$	$\mathcal{O}((n + m) \log n)$	$\mathcal{O}((n + m) \log n)$
Ehrmanntraut and Meyer [27] *	BFS	●	$\mathcal{O}(\log n)$	$\mathcal{O}(n^2 \log n) / \mathcal{O}(n^3 \log n)^\diamond$	$\mathcal{O}(n^3 \log n)$
This work $\boxplus$	Varies	●	$\mathcal{O}(n)$ to $\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$ to $\mathcal{O}(n^3 \log n)$	$\mathcal{O}(n^2)$ to $\mathcal{O}(n^3 \log n)$

● Returns path from start node to target node ● Returns predecessors ○ Only computes distances

* Protocol does not find shortest paths in general but only performs a Breadth-First-Search, i. e., all edges have length 1.

Δ Protocol designed for sparse graphs. m denotes the number of edges.

* S denotes the effort required to shuffle the distance matrix, which depends on the used primitive.

$\diamond$ The smaller communication complexity requires the primitive supporting $n \times n$ matrix multiplications with $\mathcal{O}(n^2)$ communication, such as [1, 8, 23].

$\dagger$ Protocol is affected by the leakage described in Section 4 and Appendix C.

$\ddagger$ Protocol is not data-oblivious and always leaks the number of reachable nodes.

$\boxplus$ Our protocol adapts sub-protocols to the computation's environment, and hence has different characteristics in different environments.

Protocol 1 Generic privacy-preserving Dijkstra protocol [2, 5].

```

1: function PP-DIJKSTRA ( $[L], [\vec{Start}]$ )  $\triangleright L$ : Distance matrix
2:   for  $1 \leq i \leq n$  do in parallel
3:      $[\vec{Dist}_i] \leftarrow (1 - [\vec{Start}_i]) \cdot \infty$ 
4:      $\overline{Visited}_i \leftarrow 0$ 
5:    $[\sigma] \leftarrow \text{GENERATEPERMUTATION}(n)$ 
6:   Shuffle  $[\vec{Dist}]$  and  $[L]$  using  $[\sigma]$ .
7:   for  $1 \leq \text{iter} \leq n$  do
8:      $[v] \leftarrow \text{ARGMINOFUNVISITED}([\vec{Dist}], \overline{Visited})$ 
9:      $i \leftarrow \text{REVEAL}([v])$ 
10:     $\overline{Visited}_i \leftarrow 1$ 
11:    for  $1 \leq j \leq n$  do in parallel
12:       $[\text{newDist}] \leftarrow [\vec{Dist}_i] + [L_{i,j}]$ 
13:       $[\text{isBetter}] \leftarrow [\text{newDist}] < [\vec{Dist}_j]$ 
14:       $[\vec{Dist}_j] \leftarrow [\text{isBetter}] ? [\text{newDist}] : [\vec{Dist}_j]$ 
15:   Unshuffle  $[\vec{Dist}]$  using  $[\sigma]$ .
16:   return  $[\vec{Dist}]$ 

```

sparse graphs. Their performance characteristics are listed in Table 1. On dense graphs, Anagreh et al. [5] find that the Floyd-Warshall protocol is the fastest on small graphs, and that Dijkstra protocol is faster on larger graphs. The cross-over point depends on the network setting [5]. Further, these protocols also only compute the lengths of shortest paths, but do not return any actual paths.

To the best of our knowledge, the only two SMPC protocols [12, 27] that create paths are not general shortest path protocols, but

Breadth-First-Search (BFS) protocols, which solve the shortest path problem only for the special case that all edges have length 1. Blanton et al. [12] use the predecessors from their BFS protocol to begin at the target node and iteratively step towards the start node. Fake nodes are added to hide the path's number of steps. An optimized version of this protocol is used as baseline in this paper and discussed in more depth in Section 5.2.

Ehrmanntraut and Meyer [27] follow a different approach: Their BFS encodes the predecessor information as matrix, allowing them to use the square-and-multiply algorithm to step from the target node to the start node in $\mathcal{O}(\log n)$ rounds. In Section 5.2.2, we re-use this protocol as one instantiation of a sub-functionality.

Finally, general SMPC frameworks to adapt message-push algorithms into privacy-preserving protocols [9, 38, 42] should be mentioned, even though, to the best of our knowledge, they have not yet been used to find shortest paths.

Furthermore, other approaches to privacy-preserving shortest paths without SMPC exist in the context of cloud outsourcing: GraphOS [19] leverages a trusted execution environment. It hides memory access patterns from the host system using an ORAM, leading to significant overheads. PathGES [28] uses searchable encryption to store an encrypted graph on an untrusted cloud provider in a way that allows shortest path queries. This requires one party to have full knowledge of the graph, and some information about the graph structure and the queries is leaked to the cloud provider.

3.2 Non-privacy preserving shortest paths

Whilst Dijkstra's algorithm [24] has been shown to be an optimal sequential algorithm on general graphs [16], efficient parallel shortest path algorithms have proven themselves to be much more

challenging to construct. Algorithms such as [36, 47] were able to reduce the time required to compute shortest paths, but this came at the cost of significant overheads in the total required work. Only recently, two independent breakthroughs [16, 46] managed to parallelly compute shortest paths without significantly increasing the computational overhead by leveraging parallel approximative shortest path algorithms and iteratively improving the approximation quality until an exact solution can be determined.

However, these algorithms are not suitable as basis for SMPC protocols due to two reasons: As already noted by Anagreh et al. [5], the old exact parallel algorithms dynamically create and join threads, making privacy-preserving adaptations highly inefficient. This also applies to newer parallel approximative shortest path algorithms like [17, 45]. Further, reductions of exact to approximative shortest paths require non-integer arithmetic that is able to accurately represent values like $\ln\left(\frac{3}{2}\right)$. This adds another significant overhead to potential SMPC implementations [32].

3.3 Hybrid compilers

In this work, we evaluate different protocols for realizing sub-functionalities like demultiplexes, and switch between sub-protocols depending on the best approach for any given input size, network delay and bandwidth, and security requirements. This is similar but orthogonal to the approach of hybrid compilers, which enable switching between SMPC primitives with the goal of using the fastest primitive for each basic operation of the evaluated protocol [15, 20, 29]. In comparison, this paper assumes a fixed underlying primitive, and instead adjusts the protocol to the execution setting, i. e., our optimizations are made at a higher level.

Theoretically, it should be possible to integrate our high-level optimizations and the low-level optimizations done by hybrid compilers. However, this complicates the already complex optimization problems solved by hybrid compilers, and lacks software support in available frameworks.

4 Leakages in prior protocols

In this section, we briefly describe a subtle leakage in the prior shortest path protocols based on the Dijkstra algorithm [2, 5]. As described in Section 3.1, both protocols first shuffle the nodes in an initialization phase. In each iteration, they then find the unvisited node closest to the start node and reveal its (shuffled) index, which allows efficient updates of the neighbors' distances.

It is claimed that the revealed indices do not reveal any private information due to shuffling of the nodes [2, 5]. In a follow-up paper, Anagreh and Laud [4] specify that the revealed indices can be simulated using a random permutation of $1, \dots, n$.

We find that this is not true when multiple nodes have the same distance to the start node. In that case, both protocols choose the closest unvisited node with the lowest shuffled index, as explained in detail in Appendix C. As a result, the revealed indices always increase when the distance to the start node does not increase.

It follows that, on graphs where multiple nodes have the same distance to the start node, the revealed (shuffled) node indices are not uniformly random permutations of $1, \dots, n$. This breaks the simulator by Anagreh and Laud [4]. Further, some bounds on the

number of layers in the graph and the size of each layer can be inferred from this leakage.

To fix the leakage, the protocols can select the node with the lowest unshuffled index when there are multiple nodes with the same distance to the start node. This ensures that the shuffling permutation cannot influence the node selection, and the revealed (shuffled) indices truly become a uniformly random permutation of $1, \dots, n$. An efficient implementation of this fix, and a formal proof of its security, are presented in Appendix C.

5 Building shortest paths

This section presents our single-pair shortest path protocol, along a baseline protocol. As described in Functionality 1, this protocol takes the number of nodes n as public input, and a graph's distance matrix as well as the start and target node as secret inputs. The start node and target node are encoded as unit vectors, i. e., as vectors only containing zeroes except a single one at the position of the node index³.

Public input: The number of nodes n .

Secret inputs:

A $n \times n$ distance matrix $[L]$, and

a unit vector $\begin{bmatrix} E^{s,n} \end{bmatrix}$ of length n encoding the start node s , and

a unit vector $\begin{bmatrix} E^{t,n} \end{bmatrix}$ of length n encoding the target node t .

Secret output:

A $n \times n$ matrix $[S^p]$ that encodes a shortest path $p = (v_1, \dots, v_k)$ from s to t in the following way: For all $1 \leq i \leq k$, $S_{i,i}^p = \overline{E^{v_i,n}}$, and $S_{i,i}^p = 0^n$ for all $k < i \leq n$. If no path from s to t exists, all entries of S^p are zero.

Functionality 1: The single-pair shortest path functionality.

The functionality has only a single secret output, which is a matrix that encodes a shortest path from the start to the target node. The i 'th row of this matrix corresponds to the unit vector representation of the i 'th visited node. Note that Functionality 1 only requires the resulting path to be a shortest path. This means that the protocol is able to freely choose any shortest path when multiple shortest paths from s to t exist.

As shown in Figure 2, we dissect this functionality into two sub-functionalities. The first sub-functionality determines whether a path from the start to the target node exists, and calculates the "successors". These successors are pointers to the next node on the shortest path, which then used by the second path-construction sub-functionality that returns the shortest path.

Both sub-functionalities operate independently of each other, i. e., the path construction is compatible with any successor calculation sub-protocol that produces valid successors. This allows us to find multiple sub-protocols for each sub-functionality, and then adaptively use the fastest sub-protocol based on the number of nodes, the network setting and used SMPC primitive.

In the following, Section 5.1 describes the modification of two prior (length of) shortest path protocols to achieve two successor

³Recall that we assume that $V = \{1, \dots, n\}$.

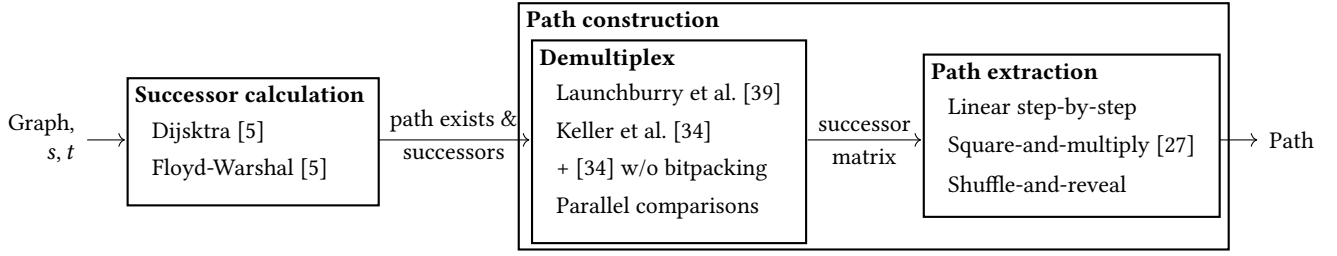


Figure 2: Our approach to finding the single-pair shortest path.

calculation sub-protocols. We then present a baseline path construction and our optimized protocol Section 5.2. It should be noted that both protocols profit from optimized successor calculations.

Finally, it should be noted that Appendix B also discusses a different approach to realizing Functionality 1 that is inspired by the min-addition algorithm of Berge [11]. This protocol contains beautiful optimizations and requires only $\mathcal{O}(\log^2 n)$ rounds on graphs with n nodes, but our evaluation in Section 6 will show that it is always slower than the protocol presented in this section.

5.1 Successor calculation

Readers familiar with shortest path algorithms might be surprised that our protocol is computing successor nodes, as shortest path algorithms usually compute predecessors. These predecessors point to the previous node in the shortest path, which traditionally is built by starting at the target node and following the predecessors until the start node is reached. However, in our protocol, this approach would lead to issues during the path construction since the path may have fewer than $n - 1$ steps, which is hidden by adding “null” nodes as padding. When building the path back-to-front, this padding has to be added afterward while keeping the number of steps hidden. We have not found a way to do so without unacceptable overheads.

Instead, we propose a trick that allows computing the successors without any overhead compared to calculating the predecessors. This allows building the path front-to-back, and obviously adding the padding becomes much easier and performant.

The trick works by applying prior shortest path protocols “backwards”, i. e., by reversing all edges and then starting the shortest path search from the target node. Figure 3 visualizes this transformation on the example graph in Figure 3a. Its edges are flipped, resulting in the graph in Figure 3c. Performing a shortest path search on that transformed graph starting on the target node t results in the “predecessors” in Figure 3d, which equal the successors of the shortest paths ending in t in the original graph G . Appendix D.1 formally proves the correctness of this trick.

Compared to a protocol computing the predecessors, this trick does not introduce any overheads as it only transposes the distance matrix. This is done implicitly by changing accessed indices, for example, in Line 3 of Protocol 2 or Line 16 of Protocol 3.

While this trick allows our protocol to compute successors instead of predecessors, the successor calculation still has to be added to the prior shortest path protocols. In the case of the Floyd-Warshall protocol by Anagreh et al. [5], a well-known technique to compute predecessors in the classical (non-privacy preserving) setting can

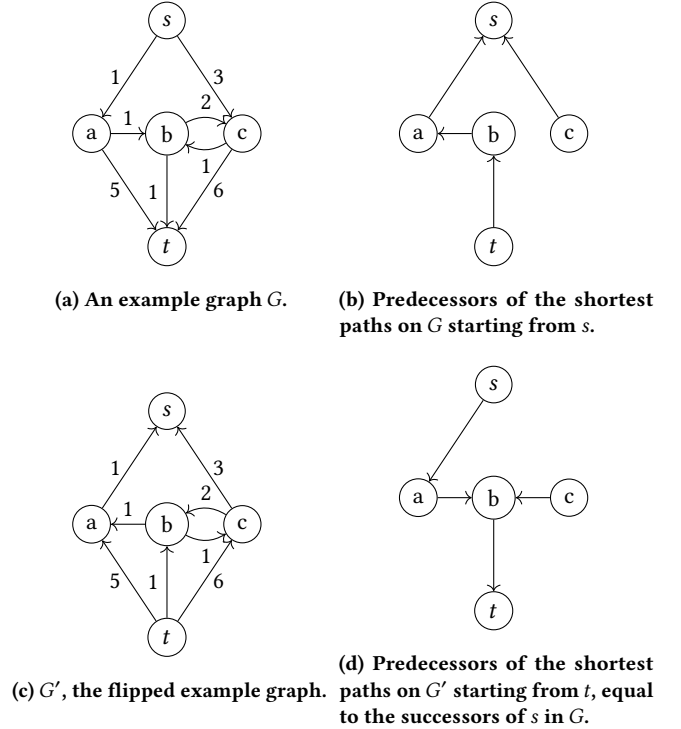


Figure 3: A visualization of the trick used to compute successors instead of predecessors.

simply be inserted, resulting in Protocol 2. Note that we deliberately store the successors as numerical indices instead of unit-vectors to ensure that this change does not affect the communication complexity of the protocol⁴. Further, it is determined whether the graph actually contains a path from the start node to the target node by comparing the length of the path with $\frac{\infty}{2}$. This ensures correctness in edge-cases involving edges with negative lengths.

Modifying the Dijkstra protocol is more challenging, as the nodes are shuffled during the initialization, requiring additional measures to unshuffle the successors. In Protocol 3, this is done by precomputing the inverse permutation and using this precomputed information when updating the successors in Line 19. As shown in

⁴Preliminary tests revealed that the overhead would not be acceptable otherwise.

Protocol 2 The Floyd-Warshall protocol from [5] with successor computation. Modifications are highlighted in blue.

```

1: function FLOYDWARSHALL( $[L], [\overrightarrow{Start}], [\overrightarrow{End}]$ )
2:   for  $1 \leq i, j \leq n$  do
3:      $[Dist_{i,j}] \leftarrow \begin{cases} [L_{j,i}] & \text{if } i \neq j \\ [0] & \text{if } i = j \end{cases}$ 
4:      $[Succs_{i,j}] \leftarrow [i]$ 
5:   for  $1 \leq k \leq n$  do
6:     for  $1 \leq i, j \leq n$  do in parallel
7:        $[c] \leftarrow ([Dist_{i,k}] + [Dist_{k,j}] < [Dist_{i,j}])$ 
8:        $[Dist_{i,j}] \leftarrow [c] ? [Dist_{i,k}] + [Dist_{k,j}] : [Dist_{i,j}]$ 
9:        $[Succs_{i,j}] \leftarrow [c] ? [Succs_{k,j}] : [Succs_{i,j}]$ 
10:     $[hasPath] \leftarrow [\overrightarrow{End}] \cdot [Dist] \cdot [\overrightarrow{Start}]^T < \frac{\infty}{2}$ 
11:  return ( $[hasPath], [\overrightarrow{End}], [Succs]$ )

```

Appendix C, the inversed permutation can also be used to prevent a leakage that occurs in the original protocol by Anagreh et al. [5]. Hence, the precomputation for the successors computation and leakage prevention has to be done only once, minimizing the overall performance impact.

Protocol 3 The Dijkstra protocol from [5] with successor computation and leakage prevention. Modifications are highlighted in blue.

```

1: function DIJKSTRA( $[L], [\overrightarrow{Start}], [\overrightarrow{End}]$ )
2:   for  $1 \leq i \leq n$  do
3:      $[Dist_i] \leftarrow (1 - [\overrightarrow{End}_i]) \cdot \infty$ 
4:      $Visited_i \leftarrow 0$ 
5:      $[InvPerm_i] \leftarrow [i]$ 
6:      $[Succs_i] \leftarrow [i]$ 
7:    $[\sigma] \leftarrow \text{GENERATEPERMUTATION}(n)$ 
8:   Shuffle  $[Dist]$ ,  $[InvPerm]$  and  $[L]$  using  $[\sigma]$ .
9:   for  $1 \leq iter \leq n$  do
10:    for  $1 \leq j \leq n$  do ▷ Prevent leakage as in Section 4.
11:       $[Tmp_j] \leftarrow (n + 1) \cdot [Dist_j] + [InvPerm_j]$ 
12:       $[v] \leftarrow \text{ARGMINOFUNVISITED}([Tmp], Visited)$ 
13:       $i \leftarrow \text{REVEAL}([v])$ 
14:       $Visited_i \leftarrow 1$ 
15:      for  $1 \leq j \leq n$  do in parallel
16:         $[newDist] \leftarrow [Dist_i] + [L_{j,i}]$ 
17:         $[isBetter] \leftarrow [newDist] < [Dist_j]$ 
18:         $[Dist_j] \leftarrow [isBetter] ? [newDist] : [Dist_j]$ 
19:         $[Succs_j] \leftarrow [isBetter] ? [InvPerm_i] : [Succs_j]$ 
20:      Unshuffle  $[Dist]$  and  $[Succs]$  using  $[\sigma]$ .
21:       $[hasPath] \leftarrow [\overrightarrow{Start}] \cdot [Dist] < \frac{\infty}{2}$ 
22:  return ( $[hasPath], [Succs]$ )

```

An important property of both sub-protocols is that the produced predecessors do not contain any cycles, even when the input graph

has cycles with a total weight of zero. This is important to ensure the correctness of the following steps in the protocol, and is proven formally in Appendix D.2.

Finally, we initialize the successor of each node i to be node i , i. e., with a small single-node loop. As the successors of unreachable nodes and the target node remain unchanged in both protocols, this single node loop can be used by the following path construction to detect the end of the path.

5.2 Path construction

Once the successors have been computed, the next step is the extraction of the path from these successors. It may appear that this problem is as easy as iteratively following the successors. However, the simplicity is thwarted by cases in which no path from the start to the target node exists, or such a path exists but has fewer than $n - 1$ steps. Both cases must be handled obviously, i. e., without leaking any information.

We first present a baseline protocol by Blanton et al. [12], that solves these issues by adding fake nodes before ([12]'s rough equivalent of) the successor calculation. These are connected to the real nodes such that the path construction is routed to the fake nodes when it reaches the end of the real path, hence hiding the fact that the real path has ended. Protocol 4, our implementation of the baseline, has two optimizations: First, we add the fake nodes only after the successor calculation. This eliminates the fake node overhead during the successor calculation. Secondly, we notice that the main loop proposed by Blanton et al. [12] implicitly performs a demultiplex using many individual comparisons. We replace this construction by a generic demultiplex sub-functionality, and select the fastest demultiplex sub-protocol according to the number of nodes, network, and used primitive.

Protocol 4 The path construction baseline inspired by [12].

```

1: function CONSTRUCTPATH( $[Succs], [\overrightarrow{Start}], [hasPath]$ )
   Add fake nodes:
2:   for  $1 \leq i \leq n$  do in parallel
3:      $[cond] \leftarrow [Succs_i] = i$ 
4:      $[SuccsWithFake_i] \leftarrow [cond] ? [Succs_i] : n + 1$ 
5:      $[SuccsWithFake_{n+i}] \leftarrow [n + i + 1]$ 
6:      $[ResultWithFake_{1,i}] \leftarrow [hasPath] \cdot [\overrightarrow{Start}_i]$ 
7:      $[ResultWithFake_{1,n+1}] \leftarrow 1 - [hasPath]$ 
   Main loop:
8:   for  $1 \leq i \leq n - 1$  do
9:      $[ResultWithFake_{i+1,\bullet}] \leftarrow \text{DEMUX}([ResultWithFake_{i,\bullet}], [Succs], 2n)$ 
   Remove fake nodes:
10:  for  $1 \leq i, j \leq n$  do
11:     $[Result_{i,j}] \leftarrow [ResultWithFake_{i,j}]$ 
12:  return  $[Result]$ 

```

In contrast, our optimized path construction, Protocol 5, follows the approach depicted in Figure 2, which decomposes the path construction into two sub-functionalities: The first sub-functionality

demultiplexes all successors in parallel, resulting in a “successor matrix” that encodes the successors as unit vectors. Section 5.2.1 briefly reviews different demultiplex sub-protocol that can be used to do so. We again select the fastest sub-protocol according to the number of nodes, network setting and used primitive. Further, Protocol 5 efficiently forces the whole the successor matrix to be zero when no path from the start node to the target node exists, and sets the target node’s successor to zero.

Protocol 5 Our optimized path construction.

```

1: function CONSTRUCTPATH( $\vec{[Succs]}$ ,  $\vec{[Start]}$ ,  $[hasPath]$ )
2:   for  $1 \leq i \leq n$  do in parallel
3:      $[SuccMatr_{i,\bullet}] \leftarrow \text{DEMUX}(\vec{[Succs]}_i, n,$ 
                                      $cond = [hasPath])$ 
4:      $[SuccMatr_{i,i}] \leftarrow [0]$ 
5:   return EXTRACTPATH( $[SuccMat]$ ,  $\vec{[Start]}$ )
    
```

The second sub-functionality extracts the final path from the successor matrix, which can be done using different sub-protocols. We present existing sub-protocols and propose a new sub-protocol to realize this sub-functionality in Section 5.2.2.

5.2.1 Brief review of demultiplex protocols. A demultiplex is a common building block of SMPC protocols that converts a numerical index x into the corresponding unit vector $\vec{E}^{x,m}$ of length m [34, 39].

A common sub-protocol to perform such demultiplexes was introduced by Launchbury et al. [39]. It first determines the bit-decomposition of the target index x once, and then recursively demultiplexes the first half and the second half of the bits. This results in two unit vectors of size $\sqrt{m}$, which are then combined into a length- m unit vector in a single communication round. Overall, this approach requires $\mathcal{O}(\log \log m)$ communication rounds, but $\mathcal{O}(m \log \log m)$ communication and computation.

Keller et al. [34] introduced an alternative sub-protocol that also first decomposes the target index x into its bits, but then processes these bits one bit at a time. Concretely, they start with a unit-vector of size 2, based solely on the least-significant-bit and iteratively incorporate the next bit, which doubles the size of the unit-vector. This is repeated until all bits are incorporated. Crucially, Keller et al. [34] optimize this approach towards primitives operating on large fields by packing multiple bits into a single field element. This significantly reduces the communication cost of this protocol, which is $\mathcal{O}(m)$ but with a small hidden constant, at the expense of the higher round complexity of $\mathcal{O}(\log m)$. We notice that a considerable effort is spent on the final bit-decompositions required to unpack the bits. Therefore, we also investigate if this sub-protocol has its merits without bitpacking in Section 6.2.3.

Finally, the trivial approach, consisting of just m parallel comparisons, has been used implicitly in literature [12, 13]. Protocol 6 presents a slightly optimized version of this approach, based on the observation that in MP-SPDZ less-than-or-equals ($\leq$) comparisons are much faster than equals ($=$) comparisons. This will be discussed in more detail in Section 6.2.2.

Protocol 6 The optimized “parallel comparisons” demultiplex.

```

1: function DEMUXCONSTRound( $[x]$ ,  $n$ ,  $[cond]$ )
2:    $[x] \leftarrow [x] + (1 - [cond]) \cdot n$ 
3:   for  $1 \leq i \leq n$  do in parallel
4:      $\vec{[Result]}_i \leftarrow ([x] \geq i)$ 
5:   for  $1 \leq i \leq n - 1$  do
6:      $\vec{[Result]}_i \leftarrow \vec{[Result]}_i - \vec{[Result]}_{i+1}$ 
7:   return  $\vec{[Result]}$ 
    
```

5.2.2 Extracting the path from the successor matrix. Once the numerical successors have been demultiplexed into the successor matrix, multiple sub-protocols can be applied to extract the final path. The simplest amongst them is roughly equivalent to the baseline approach presented in Protocol 4, using vector-matrix operations instead of the demultiplex to follow the successors. It is further simplified by the fact that self-loops are removed from the successor matrix. This means that the successor of the last node in the path is represented by a vector containing only zeroes, which naturally leads to a padding with zero-vectors without any the need for fake nodes. Overall, this sub-protocol required $\mathcal{O}(n)$ rounds, $\mathcal{O}(n^3)$ computation, and communication between $\mathcal{O}(n^2)$ and $\mathcal{O}(n^3)$ depending on the primitive.

Protocol 7 The linear path extraction.

```

1: function SUCCESSORMATRIXTOPATH( $[Succs]$ ,  $\vec{[Start]}$ )
2:    $\vec{[Result]}_{1,\bullet} \leftarrow \vec{[Start]}$ 
3:   for  $1 \leq i \leq n - 1$  do
4:      $\vec{[Result]}_{i+1,\bullet} \leftarrow \vec{[Result]}_{i,\bullet} \cdot [Succs]$ 
5:   return  $\vec{[Result]}$ 
    
```

The round complexity can be reduced by applying a technique presented by Ehrmanntraut and Meyer [27]. They noticed that calculating powers of an adjacency matrix reveals the nodes reachable in a set number of steps. This manifests as $\vec{Result}_{i,\bullet} = \vec{Start} \cdot Succs^{i-1}$ in our case. Hence, the square-and-multiply algorithm can be used to efficiently calculate $Succs^i$ for all $0 \leq i \leq n-1$, thus extracting the path in $\mathcal{O}(\log n)$ rounds. This comes at the cost of $\mathcal{O}(n^3 \log n)$ computation and communication between $\mathcal{O}(n^2 \log n)$ and $\mathcal{O}(n^3 \log n)$ depending on the primitive.

We also present a novel sub-protocol to extract the path from the successor matrix that shuffles the nodes. After, the sub-protocol iteratively visit the nodes as in the linear Protocol 7, with the twist that the shuffled index of the visited node is revealed, thus enabling further optimizations. The main complication is that the number of steps and the case when no path exists have to be hidden.

Protocol 8 handles this similarly to the baseline by routing nodes without successors into a chain of fake nodes, which is done without any communication in Lines 2 to 7. The main loop in Lines 13 to 16 then follows this chain of fake nodes once the end of the path is reached, or no path exists in the first place. Afterward, the path is unshuffled and the fake nodes are removed by simply cutting off the associated matrix entries.

As the nodes are shuffled after adding the fake nodes, the revealed node indices are random and the revealed indices of real nodes are indistinguishable from the revealed indices of fake nodes. Hence, revealing the visited node indices does not leak any information. Appendix D.3 proves this formally.

Further, the performance characteristics of Protocol 8 are heavily influenced by the shuffling of the nodes and the unshuffling of the path, as the main loop requires only $\mathcal{O}(n)$ communication in $\mathcal{O}(n)$ rounds. The shuffling effort varies greatly depending on the used primitive, but each primitive requires at least $\Omega(n^2)$ communication since a $(2n-1) \times (2n-1)$ matrix is (un-)shuffled.

Protocol 8 The shuffle-and-reveal path extraction.

```

1: function SUCCMATRIXTOPATH( $[Succs]$ ,  $[Start]$ ,  $[hasPath]$ )
   Add fake nodes:
2:   for  $1 \leq i, j \leq n$  do ▷ Copy old successors.
3:      $[Succs'_{i,j}] \leftarrow [Succs_{i,j}]$ 
4:   for  $1 \leq i \leq n$  do ▷ Connections to fake nodes.
5:      $[Succs'_{i,n+1}] \leftarrow 1 - \sum_{j=1}^n [Succs_{i,j}]$ 
6:   for  $1 \leq i \leq n-2$  do ▷ Connections between fake nodes.
7:      $[Succs'_{n+i,n+i+1}] \leftarrow [1]$ 
   Extract the path:
8:    $[\sigma] \leftarrow \text{GENERATEPERMUTATION}(2n-1)$ 
9:    $[Tmp] \leftarrow [Start]$ , padded to length  $2n-1$  with  $[0]$ .
10:  Shuffle  $[Succs']$  and  $[Tmp]$  using  $[\sigma]$ .
11:   $current \leftarrow \text{REVEAL}(\sum_{j=1}^{2n-1} j \cdot [Tmp_j])$ 
12:   $[ResultWithFakeNodes_{1,current}] \leftarrow [hasPath]$ 
13:  for  $1 \leq step \leq n-1$  do
14:     $next \leftarrow \text{REVEAL}(\sum_{j=1}^{2n-1} j \cdot [Succs'_{current,j}])$ 
15:     $[ResultWithFakeNodes_{step+1,next}] \leftarrow [1]$ 
16:     $current \leftarrow next$ 
17:  for  $1 \leq i \leq n$  do in parallel
18:    Unshuffle  $[ResultWithFakeNodes_{i,*}]$  using  $[\sigma]$ .
   Remove fake nodes:
19:  for  $1 \leq i, j \leq n$  do
20:     $[Result_{i,j}] \leftarrow [ResultWithFakeNodes_{i,j}]$ 
21:  return  $[Result]$ 

```

6 Evaluation

In this section, we discuss the runtimes of our microbenchmarks and the total runtimes of our single-pair shortest path protocol.

6.1 Setup

We implement our protocols in MP-SPDZ [33] at the state of the git commit 5f45083f. The primitive replicated-ring [8] is used to evaluate the protocols with semi-honest security, and sy-rep-ring [1] with malicious security. These primitives only support three parties.

Further, MP-SPDZ does not separate between most primitives' offline and online phases, where pre-processed material like random

bits is generated in the offline phase, and input-dependent computations are performed in the online phase. Instead, it pre-processes on the fly. For example, when random bits are needed, it generates a batch of random bits. Unused bits are cached, and the next batch is generated once the cache is empty. We set the batch size to 1 000 000 to reflect the parallel nature of the protocols in this paper.

All computations ran in a container with access to four cores of an AMD EPYC 7702P processor clocked at 2GHz and 128 gigabyte RAM, which is shared among all three parties.

Further, NEON [37] is used to simulate three different network settings: In the LAN setting, the delay between any two parties is one millisecond (i. e., 2ms "ping"), and the bandwidth of each party is restricted to one gigabit per second. The WAN setting introduces a 10-milliseconds delay, and bandwidth restrictions of 100 megabits per second. We also evaluate without any artificial network limitations, called the unrestricted setting.

Each computation was repeated five times. All figures in this section therefore also show the 95% confidence intervals as error bands. Often, these intervals are so small that they are hardly visible.

Finally, our implementations, runtime logs, and plotting scripts are publicly available⁵.

6.2 Microbenchmarks

A substantial part of our protocols is the ability to switch sub-protocols with the goal to select the fastest sub-protocol for each input size, network setting and used primitive. Hence, microbenchmarks are required for each sub-functionality.

For brevity, we will mostly focus on the semi-honest LAN setting in this section, and only consider other settings when necessary. The runtimes in other settings are included in the artifact.

6.2.1 Successor calculations. The first prominent sub-functionality of our protocol is the successor calculation, where Section 5.1 presented two sub-protocols that modify prior (length of) shortest path protocols by Anagreh et al. [5]. The runtime of these modified sub-protocols are mostly in line with the runtimes of the original protocols. As shown in Figure 4, the Floyd-Warshall protocol is faster than the Dijkstra protocol on small graphs, but slower on larger graphs. The exact cross-over point depends on the network setting and primitive, but generally varies from 90 to 300 nodes. On graphs with 300 or more nodes, the Dijkstra protocol is faster in all evaluated settings.

This is explained by the Floyd-Warshall protocol's high communication effort (163GB at 1 000 nodes) which scales with $\mathcal{O}(n^3)$. In comparison, the Dijkstra protocol's communication scales with $\mathcal{O}(n^2)$ and requires only 414MB at 1000 nodes. This outweighs the higher round complexity of the Dijkstra protocol, which results in 99 936 rounds at 1000 nodes, compared to 11 000 rounds required by the Floyd-Warshall protocol.

6.2.2 Comparisons. Beyond the explicitly stated sub-functionalities, our (sub-)protocols commonly use comparisons. As MP-SPDZ derives all types of comparisons from less-than ($<$) and equals ($=$), for example by $a \leq b = 1 - (b < a)$, we evaluate both types by repeatedly running a set of parallel comparisons for a hundred times. Figure 5 presents the resulting average runtime per set of

⁵<https://anonymous.4open.science/r/shortest-path-771B>

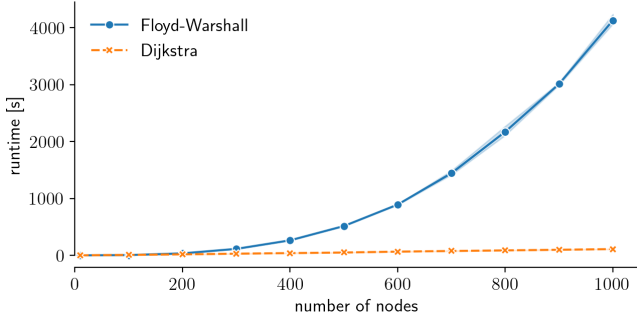


Figure 4: Runtimes of the successor calculation using the semi-honest primitive in the LAN setting.

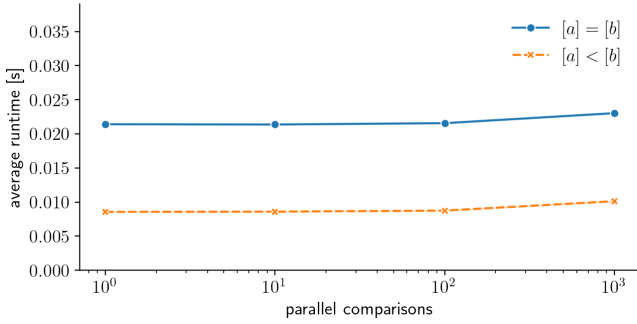


Figure 5: Runtimes of comparisons using the semi-honest primitive in the LAN setting.

parallel comparisons, i. e., the total runtime divided by 100. We find that less-than comparisons are, on average, 2.47 times faster than equals comparisons. This is consistent across network settings and primitives, with less-than being 2.08 times faster than equals using the malicious security primitive. This is explained by equals requiring 20 rounds using the semi-honest primitive (malicious security 21 rounds), whereas less-than requires only 8 rounds (malicious security 9 rounds).

6.2.3 Demultiplexes. The demultiplex protocols presented in Section 5.2.1 were evaluated like the comparisons. Figure 6 presents the resulting average times per demultiplex in the LAN setting using the semi-honest primitive. A noticeable result is that Keller et al. [34]’s protocol is outperformed by the alternative protocols. This is caused by the bit-decompositions needed to “unpack” the bits. Their high usage of pre-processed material leads to more rounds and communication than the protocol’s counterpart without bitpacking.

Further, the presented runtimes generally closely follow the number of rounds the protocols require⁶, which means that their bottleneck is the communication delay. The only exception is the “parallel comparisons” protocol. It constantly requires 8 rounds, but its runtime increases linearly with the number of nodes. This is explained by its linearly increasing communication. Still, the parallel comparisons protocol is the fastest protocol in this setting.

⁶All rounds and communication numbers are included in the artifact.

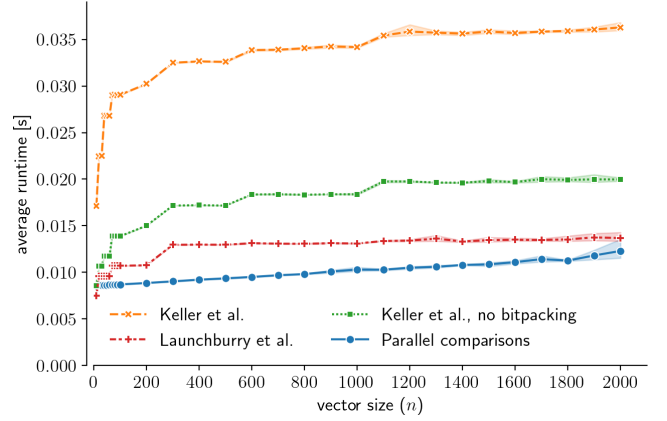


Figure 6: Runtimes of single demultiplexes by the vector size using the semi-honest primitive in the LAN setting.

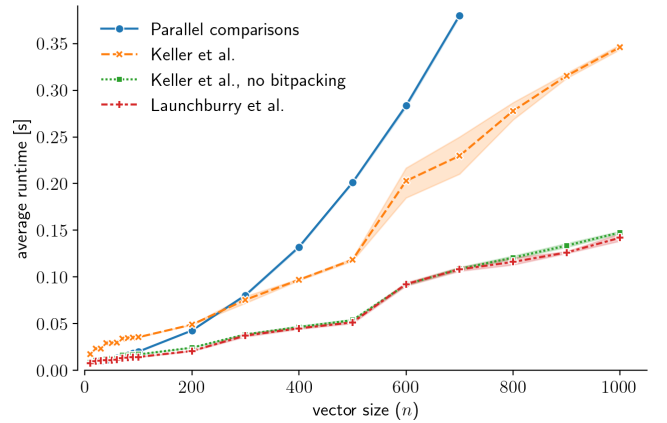


Figure 7: Runtimes of n parallel demultiplexes by the vector size (n) using the semi-honest primitive in the LAN setting.

This changes when performing n parallel demultiplexes. As presented in Figure 7, the parallel comparisons protocol is slower than its competitors. It is bottlenecked by its communication. For example, it requires 115.1 megabytes of communication at 1000 nodes, Launchburry et al. [39] requires only 24.8 megabytes. The fastest protocol is either Launchburry’s et al. [39] or Keller et al. [34] without bitpacking depending on the number of nodes. Launchburry et al. [39]’s protocol requires fewer rounds, e. g., 12 rounds vs 17 rounds at $n = 1\,000$, but more communication, e. g., 27.6 megabyte vs. 24.8 megabyte. As both protocols have similar runtimes, it seems like both protocols provide good trade-offs between rounds and communication, with the setting dictating whether a slight focus on the rounds or on the communication is advantageous.

6.2.4 Path extraction. After the demultiplexes, our protocol’s next step is the extraction of the shortest path. Section 5.2.2 presented three different sub-protocol to do so. Figure 8 displays their runtimes in the semi-honest LAN setting, where the linear protocol is the fastest from 100 until 800 nodes. On larger graphs, it is outperformed by the shuffle-reveal protocol. This is noteworthy as the

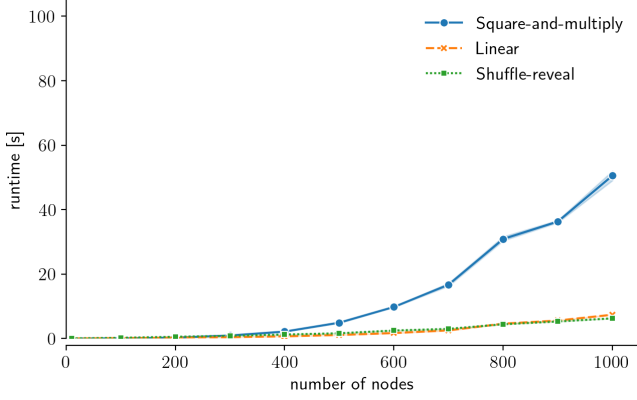


Figure 8: Runtimes of the path extraction protocols using the semi-honest primitive in the LAN setting.

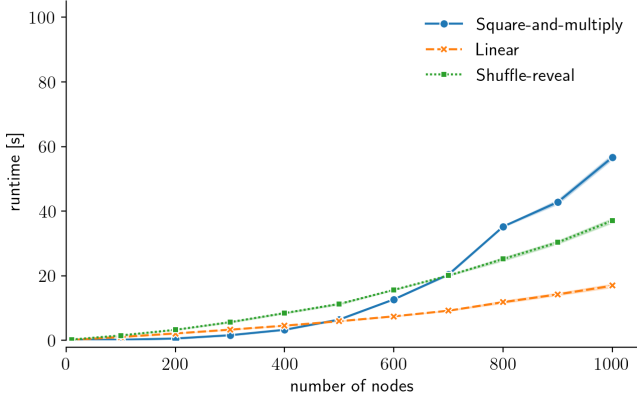


Figure 9: Runtimes of the path extraction protocols using the semi-honest primitive in the WAN setting.

shuffle-reveal protocol requires both more communication rounds (1 020 vs. 1 000 at 1000 nodes) and more communication (480MB vs. 24MB) than the linear protocol. This suggests that the linear protocol is bottlenecked by its $\mathcal{O}(n^3)$ computation, which doesn't apply to the shuffle-reveal protocol requiring only $\mathcal{O}(n^2)$ computation. The computation bottleneck also explains why the square-and-multiply protocol only is advantageous for graphs with less than 100 nodes, as it requires $\mathcal{O}(n^3 \log n)$ computation.

Consequently, it is not surprising that the square-and-multiply protocol is the fastest for up to 400 nodes in the WAN setting as depicted in Figure 9. In this setting, the communication time has a bigger influence than the computation time, leading to the linear protocol always being faster than the shuffle-reveal protocol.

6.3 Start-to-end runtimes

Using these microbenchmarks, the complete single-pair-shortest path protocol can be assembled, which accepts a secret graph and two nodes as input and produces a secret path as output. We evaluated our protocol alongside the baseline described in Section 5.2. It should be noted that we also apply the microbenchmarks to the

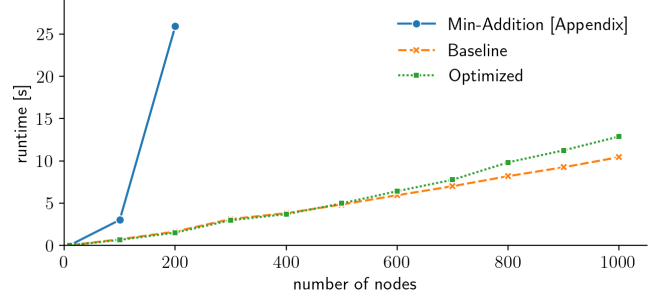


Figure 10: Runtimes of the complete shortest path protocols using the semi-honest primitive in the unrestricted network setting.

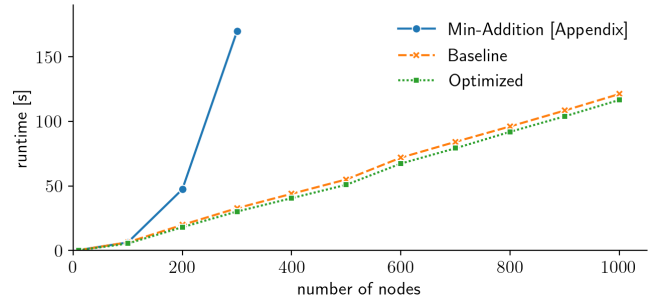


Figure 11: Runtimes of the complete shortest path protocols using the semi-honest primitive in the LAN setting.

baseline and switch the demultiplex sub-protocol depending on the evaluation setting. Hence, we compare our optimized protocol to an already-optimized baseline. Finally, we also evaluate the min-addition protocol described in Appendix B.

As shown in Figures 10 and 11, the min-addition protocol is much slower than our optimized and baseline protocols. This is explained by its high communication and computation complexity of $\mathcal{O}(n^3 \log n)$. For example, on graphs with 200 nodes, the min-addition protocol requires over 7.7 gigabytes of communication, whereas the baseline protocol requires only 28 megabytes of communication. This outweighs the fact that the min-addition protocol requires only 836 rounds, where the baseline requires 18 163 rounds.

The comparison of the baseline with our optimized protocol is more interesting and requires us to look at other network settings too: In the unrestricted network setting, the baseline outperforms our optimized protocol on graphs larger than 500 nodes, as shown in Figure 10. As the runtime in the unrestricted setting mainly depends on the computational effort, this means that the direct step matrix calculation is computationally more efficient than the detour over the successor matrix. This shows even when the baseline protocol and the optimized protocol (with shuffle-reveal path extraction) have the same asymptotic computation complexity of $\mathcal{O}(n^2)$.

However, as soon as a small delay is introduced as in the LAN setting, our optimized protocol outperforms the baseline. As shown in both Figure 11 and Table 2, the optimized protocol is faster than the baseline on all evaluated graph sizes. This makes a significant

Table 2: Comparison of the complete shortest path protocol runtimes with baseline and the optimized path construction in the LAN setting using the semi-honest primitive.

n	Baseline	Optimized	Speedup	
	s	s	s	%
100	6.24	5.31	0.93	14.97
200	19.74	18.01	1.73	8.78
300	32.70	30.11	2.58	7.90
400	43.85	40.48	3.37	7.69
500	54.98	50.84	4.14	7.53
600	71.88	67.29	4.59	6.38
700	84.14	79.16	4.98	5.92
800	96.08	91.91	4.18	4.35
900	108.42	103.92	4.50	4.15
1000	121.21	116.62	4.59	3.78

Table 3: Comparison of the complete shortest path protocol runtimes with baseline and the optimized path construction in the WAN setting using the semi-honest primitive.

n	Baseline	Optimized	Speedup	
	s	s	s	%
100	28.46	19.35	9.11	32.02
200	112.52	94.31	18.21	16.18
300	309.13	282.36	26.76	8.66
400	413.20	378.31	34.89	8.44
500	518.17	475.77	42.40	8.18
600	679.47	628.53	50.94	7.50
700	795.50	735.85	59.65	7.50
800	910.91	842.19	68.73	7.54
900	1 027.46	949.61	77.85	7.58
1000	1 144.55	1 058.25	86.30	7.54

difference: On small graphs with 100 nodes, the overall computation of the shortest path, which includes the successor calculation, is almost 15% faster than the baseline. On graphs with at least 500 nodes, our optimized protocol is roughly four to five seconds faster than the baseline. This results in diminishing procentual speedups as the total grows with the graph size. Still, our optimized protocol is 3.78% faster than the baseline at 1000 nodes.

As shown in Table 3, the difference is larger in the slower WAN network. On small graphs with 100 nodes, our optimized protocol is 32% faster than the baseline. Even on large graphs with 500 to 1000 nodes, our protocol maintains a 7.5% speedup over the baseline.

These runtimes imply that our optimized protocol is able to find better trade-offs between rounds, communication, and computation. This trade-off always results in slightly fewer rounds than the baseline, e.g., 100 968 rounds compared to 108 951 at 1000 nodes in the LAN setting. This, however, does not mean that our protocol strictly minimizes the rounds. For example, on large graphs it often prefers the linear and shuffle-reveal path extractions over the

square-and-multiply approach, which requires fewer rounds but more communication and computation.

Another interesting trade-off occurs in the LAN setting on graphs with at least 800 nodes, where the optimized protocol uses the shuffle-reveal path extraction. This results in more communication than the baseline, e.g., 1.7 vs. 1.4 gigabyte at 1000 nodes, but reduces the computation complexity from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$. In other words, the protocol trades-off faster computation for more communication in the LAN setting. In the WAN setting, where the network is slower, it does not make that trade-off and sticks with the linear path extraction that requires less communication.

7 Conclusion

In this work, we’ve presented a novel single-pair shortest path protocol which not only computes the length of the shortest path, but that also produces the shortest path by modifying prior (length of) shortest path protocols to produce successor nodes and constructing a path based on these successors. The path construction is improved by decomposing it into demultiplex and path extraction sub-functionalities. Multiple sub-protocols are presented for each sub-functionality and adaptively selected based on the number of nodes, network setting and used SMPC primitive.

Compared to a path construction baseline that follows a similar approach but that is less flexible, our evaluation has shown that our protocol is able to find better trade-offs between the number of rounds, the total communication, and computation. This results in speedups of up to 32% in the overall runtime. Even in settings where the successor calculation dominates the path construction, our optimized path construction yields speedups around 4%.

Beyond these improvements to privacy-preserving shortest path protocols, all our microbenchmarks have shown that, for many problems, there is no “dominant protocol” that is always faster than its alternatives. Instead, the optimal choice is highly dependent on the computation’s environment. This calls for a paradigm shift in protocol design: Especially as SMPC is moving towards real-world usages, it is no longer good enough to focus on optimizing the round complexity. Instead, protocols crafted for a specific environment need to be evaluated in that environment, and general protocols must provide the flexibility to adapt to their environment.

References

- [1] Mark Abspoel, Anders Dalskov, Daniel Escudero, and Ariel Nof. 2021. An Efficient Passive-to-Active Compiler for Honest-Majority MPC over Rings. In *Applied Cryptography and Network Security*, Kazuo Sako and Nils Ole Tippenhauer (Eds.). Springer International Publishing, Cham. doi:10.1007/978-3-030-78375-4_6
- [2] Abdelrahman Aly and Sara Cleemput. 2022. A Fast, Practical and Simple Shortest Path Protocol for Multiparty Computation. In *Computer Security – ESORICS 2022 (Lecture Notes in Computer Science)*. Springer Nature Switzerland, Cham. doi:10.1007/978-3-031-17143-7_37
- [3] Abdelrahman Aly and Mathieu Van Vyve. 2015. Securely Solving Classical Network Flow Problems. In *Information Security and Cryptology 2014 (Lecture Notes in Computer Science)*. Springer International Publishing, Cham. doi:10.1007/978-3-319-15943-0_13
- [4] Mohammad Anagreh and Peeter Laud. 2025. Security Proof of Single-Source Shortest Distance Protocols Built on Secure Multiparty Computation Protocols. *Cryptography* 9, 1 (March 2025), 1. doi:10.3390/cryptography9010001
- [5] Mohammad Anagreh, Peeter Laud, and Eero Vainikko. 2021. Parallel Privacy-Preserving Shortest Path Algorithms. *Cryptography* 5, 4 (2021). doi:10.3390/cryptography5040027
- [6] Mohammad Anagreh, Peeter Laud, and Eero Vainikko. 2022. Privacy-Preserving Parallel Computation of Shortest Path Algorithms with Low Round Complexity..

- In *8th International Conference on Information Systems Security and Privacy*. SCITEPRESS. doi:10.5220/0010775700003120
- [7] Mohammad Anagreh, Eero Vainikko, and Peeter Laud. 2021. Parallel Privacy-Preserving Shortest Paths by Radius-Stepping. In *2021 29th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*. 276–280. doi:10.1109/PDP52278.2021.00051
 - [8] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. 2016. High-Throughput Semi-Honest Secure Three-Party Computation with an Honest Majority. In *2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*. ACM, New York, NY, USA. doi:10.1145/2976749.2978331
 - [9] Toshinori Araki, Jun Furukawa, Kazuma Ohara, Benny Pinkas, Hanan Rosemarin, and Hikaru Tsuchida. 2021. Secure Graph Analysis at Scale. In *2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21)*. ACM, New York, NY, USA. doi:10.1145/3460120.3484560
 - [10] Richard Bellman. 1958. On a Routing Problem. *Quart. Appl. Math.* 16, 1 (1958), 87–90. doi:10.1090/qam/102435
 - [11] Claude Berge. 1962. The Theory of Graphs and Its Applications. Methuen, 130–140.
 - [12] Marina Blanton, Aaron Steele, and Mehrdad Alisagari. 2013. Data-Oblivious Graph Algorithms for Secure Computation and Outsourcing. In *8th ACM SIGSAC Symposium on Information, Computer and Communications Security (ASIA CCS '13)*. ACM, New York, NY, USA. doi:10.1145/2484313.2484341
 - [13] Enrico Bottazzi, Chan Nam Ngo, and Masato Tsutsumi. 2024. Multilateral Trade Credit Set-off in MPC via Graph Anonymization and Network Simplex.
 - [14] R. G. Busacker and P. J. Gowen. 1960. *A Procedure for Determining a Family of Minimum-Cost Network Flow Patterns*. Technical Report AD249662. Operations Research Office, Johns Hopkins Univ., Bethesda, Md.
 - [15] Niklas Büscher, Daniel Demmler, Stefan Katzenbeisser, David Kretzmer, and Thomas Schneider. 2018. HyCC: Compilation of Hybrid Protocols for Practical Secure Computation. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*. Association for Computing Machinery, New York, NY, USA, 847–861. doi:10.1145/3243734.3243786
 - [16] Nairen Cao and Jeremy T. Fineman. 2023. Parallel Exact Shortest Paths in Almost Linear Work and Square Root Depth. In *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. Society for Industrial and Applied Mathematics, 4354–4372. doi:10.1137/1.9781611977554.ch166
 - [17] Nairen Cao, Jeremy T. Fineman, and Katina Russell. 2020. Efficient Construction of Directed Hopsets and Parallel Approximate Shortest Paths. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing (STOC 2020)*. Association for Computing Machinery, New York, NY, USA, 336–349. doi:10.1145/3357713.3384270
 - [18] Octavian Catrina and Sebastian de Hoogh. 2010. Improved Primitives for Secure Multiparty Integer Computation. In *Security and Cryptography for Networks*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-642-15317-4_13
 - [19] Javad Ghareh Chamani, Ioannis Demertzis, Dimitrios Papadopoulos, Charalampos Papamanthou, and Rasool Jalili. 2023. GraphOS: Towards Oblivious Graph Processing. *Proceedings of the VLDB Endowment* 16, 13 (2023). doi:10.14778/3625054.3625067
 - [20] Edward Chen, Jinhao Zhu, Alex Ozdemir, Riad S. Wahby, Fraser Brown, and Wenting Zheng. 2023. Silph: A Framework for Scalable and Accurate Generation of Hybrid MPC Protocols. In *2023 IEEE Symposium on Security and Privacy (SP)*. 848–863. doi:10.1109/SP46215.2023.10179397
 - [21] Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. 2018. SPDZk Efficient MPC Mod 2^k for Dishonest Majority. In *Advances in Cryptology – CRYPTO 2018*. Springer International Publishing, Cham. doi:10.1007/978-3-319-96881-0_26
 - [22] Ivan Damgård and Jesper Buus Nielsen. 2003. Universally Composable Efficient Multiparty Computation from Threshold Homomorphic Encryption. In *Advances in Cryptology – CRYPTO 2003*, Dan Boneh (Ed.). Springer, Berlin, Heidelberg, 247–264. doi:10.1007/978-3-540-45146-4_15
 - [23] Ivan Damgård and Jesper Buus Nielsen. 2007. Scalable and Unconditionally Secure Multiparty Computation. In *Advances in Cryptology – CRYPTO 2007*, Alfred Menezes (Ed.). Springer, Berlin, Heidelberg. doi:10.1007/978-3-540-74143-5_32
 - [24] E. W. Dijkstra. 2022. A Note on Two Problems in Connexion with Graphs. In *Edsger Wybe Dijkstra: His Life, Work, and Legacy* (1 ed.). Vol. 45. ACM, New York, NY, USA.
 - [25] Ding-Zhu Du, Panos Pardalos, Xiaodong Hu, and Weili Wu. 2022. Primal-Dual Methods and Minimum Cost Flow. In *Introduction to Combinatorial Optimization*, Ding-Zhu Du, Panos M. Pardalos, Xiaodong Hu, and Weili Wu (Eds.). Springer International Publishing, Cham. doi:10.1007/978-3-031-10596-8_7
 - [26] Hendrik Eriksen, Marcel Keller, Claudio Orlandi, Pille Pullonen, Joonas Puura, and Mark Simkin. 2020. Use Your Brain! Arithmetic 3PC for Any Modulus with Active Security: 1st Conference on Information-Theoretic Cryptography, ITC 2020. In *1st Conference on Information-Theoretic Cryptography (Leibniz International Proceedings in Informatics)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ITC.2020.5
 - [27] Vincent Ehrmanntraut and Ulrike Meyer. 2024. Privacy-Preserving Breadth-First-Search and Maximal Flow. In *22nd Workshop on Privacy in the Electronic Society*. ACM. doi:10.1145/3689943.3695041
 - [28] Francesca Falzon, Esha Ghosh, Kenneth G. Paterson, and Roberto Tamassia. 2024. PathGES: An Efficient and Secure Graph Encryption Scheme for Shortest Path Queries.
 - [29] Vivian Fang, Lloyd Brown, William Lin, Wenting Zheng, Aurojit Panda, and Raluca Ada Popa. 2022. CostCO: An Automatic Cost Modeling Framework for Secure Multi-Party Computation. In *2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P)*. 140–153. doi:10.1109/EuroSP53844.2022.00017
 - [30] Robert W. Floyd. 1962. Algorithm 97: Shortest Path. *Commun. ACM* 5, 6 (1962), 345. doi:10.1145/367766.368168
 - [31] Craig Gentry. 2009. Fully Homomorphic Encryption Using Ideal Lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing (STOC '09)*. Association for Computing Machinery, New York, NY, USA, 169–178. doi:10.1145/1536414.1536440
 - [32] Brett Hemenway, Steve Lu, Rafail Ostrovsky, and William Welser IV. 2016. High-Precision Secure Computation of Satellite Collision Probabilities. In *Security and Cryptography for Networks*. Springer International Publishing, Cham. doi:10.1007/978-3-319-44618-9_9
 - [33] Marcel Keller. 2020. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *2020 ACM SIGSAC Conference on Computer and Communications Security (CCS '20)*. ACM, New York, NY, USA. doi:10.1145/3372297.3417872
 - [34] Marcel Keller, Emmanuela Orsini, Dragos Rotaru, Peter Scholl, Eduardo Soria-Vazquez, and Srinivas Vivek. 2017. Faster Secure Multi-party Computation of AES and DES Using Lookup Tables. In *Applied Cryptography and Network Security*, Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi (Eds.). Springer International Publishing, Cham. doi:10.1007/978-3-319-61204-1_12
 - [35] Marcel Keller and Peter Scholl. 2014. Efficient, Oblivious Data Structures for MPC. In *Advances in Cryptology – ASIACRYPT 2014 (Lecture Notes in Computer Science)*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-662-45608-8_27
 - [36] Philip N Klein and Sairam Subramanian. 1997. A Randomized Parallel Algorithm for Single-Source Shortest Paths. *Journal of Algorithms* 25, 2 (Nov. 1997), 205–220. doi:10.1006/jagm.1997.0888
 - [37] Andreas Klinger, Vincent Ehrmanntraut, and Ulrike Meyer. 2024. Estimating the Runtime and Global Network Traffic of SMPC Protocols. In *Fourteenth ACM Conference on Data and Application Security and Privacy (CODASPY '24)*. ACM, New York, NY, USA. doi:10.1145/3626232.3653258
 - [38] Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, and Bhavish Raj Gopal. 2024. Graphiti: Secure Graph Computation Made More Scalable. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. ACM, Salt Lake City, UT, USA. doi:10.1145/3658644.3670393
 - [39] John Launchbury, Iavor S. Diatchki, Thomas DuBuisson, and Andy Adams-Moran. 2012. Efficient Lookup-Table Protocol in Secure Multiparty Computation. In *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming (ICFP '12)*. ACM, New York, NY, USA, 189–200. doi:10.1145/2364527.2364556
 - [40] Chang Liu, Xiao Shaun Wang, Kartik Nayak, Yan Huang, and Elaine Shi. 2015. OblivM: A Programming Framework for Secure Computation. In *2015 IEEE Symposium on Security and Privacy*. doi:10.1109/SP.2015.29
 - [41] Thomas Loruenser, Florian Wohnner, and Stephan Krenn. 2022. A Verifiable Multiparty Computation Solver for the Assignment Problem and Applications to Air Traffic Management. In *2022 on Cloud Computing Security Workshop (CCSW'22)*. ACM. arXiv:2205.03048 [cs]
 - [42] Kartik Nayak, Xiao Shaun Wang, Stratis Ioannidis, Udi Weinsberg, Nina Taft, and Elaine Shi. 2015. GraphSC: Parallel Secure Computation Made Easy. In *2015 IEEE Symposium on Security and Privacy*. IEEE. doi:10.1109/SP.2015.30
 - [43] Benjamin Ostrovsky. 2024. Privacy-Preserving Dijkstra. In *Advances in Cryptology – CRYPTO 2024*. Springer Nature Switzerland, Cham. doi:10.1007/978-3-031-68400-5_3
 - [44] Mike Rosulek and Lawrence Roy. 2021. Three Halves Make a Whole? Beating the Half-Gates Lower Bound for Garbled Circuits. In *Advances in Cryptology – CRYPTO 2021*, Tal Malkin and Chris Peikert (Eds.). Springer International Publishing, Cham. doi:10.1007/978-3-030-84242-0_5
 - [45] Václav Rozhoň, Michael Elkin, Christoph Grunau, and Bernhard Haeupler. 2022. Deterministic Low-Diameter Decompositions for Weighted Graphs and Distributed and Parallel Applications. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. 1114–1121. doi:10.1109/FOCS54457.2022.00107
 - [46] Václav Rozhoň, Bernhard Haeupler, Anders Martinsson, Christoph Grunau, and Goran Zuzic. 2023. Parallel Breadth-First Search and Exact Shortest Paths and Stronger Notions for Approximate Distances. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing (STOC 2023)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3564246.3585235
 - [47] Thomas H. Spencer. 1997. Time-Work Tradeoffs for Parallel Algorithms. *J. ACM* 44, 5 (Sept. 1997), 742–778. doi:10.1145/265910.265923
 - [48] Tomas Toft. 2009. Solving Linear Programs Using Multiparty Computation. In *Financial Cryptography and Data Security*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-642-03549-4_6

- [49] Stefan Wüller. 2018. *Privacy-Preserving Electronic Bartering*. Ph. D. Dissertation. RWTH Aachen University. doi:10.18154/RWTH-2018-222969
- [50] Stefan Wüller, Benjamin Assadsolimani, Ulrike Meyer, and Susanne Wetzel. 2018. Privacy-Preserving Subgraph Checking. In *16th Annual Conference on Privacy, Security and Trust (PST)*. doi:10.1109/PST.2018.8514182
- [51] Stefan Wüller, Ulrike Meyer, and Susanne Wetzel. 2017. Towards Privacy-Preserving Multi-party Bartering. In *Financial Cryptography and Data Security*, Michael Brenner, Kurt Rohloff, Joseph Bonneau, Andrew Miller, Peter Y.A. Ryan, Vanessa Teague, Andrea Bracciali, Massimiliano Sala, Federico Pin-tore, and Markus Jakobsson (Eds.). Springer International Publishing, Cham. doi:10.1007/978-3-319-70278-0_2
- [52] Stefan Wüller, Wadim Pessin, Ulrike Meyer, and Susanne Wetzel. 2016. Privacy-Preserving Two-Party Bartering Secure against Active Adversaries. In *14th Annual Conference on Privacy, Security and Trust*. doi:10.1109/PST.2016.7906968
- [53] Stefan Wüller, Michael Vu, Ulrike Meyer, and Susanne Wetzel. 2017. Using Secure Graph Algorithms for the Privacy-Preserving Identification of Optimal Bartering Opportunities. In *2017 on Workshop on Privacy in the Electronic Society (WPES '17)*. ACM, New York, NY, USA. doi:10.1145/3139550.3139557
- [54] Andrew C. Yao. 1982. Protocols for Secure Computations. In *23rd Annual Symposium on Foundations of Computer Science*. IEEE. doi:10.1109/SFCS.1982.38

A Application to privacy-preserving bartering

In this section, we show an example application that can greatly profit from our shortest path protocol presented in Section 5, namely privacy-preserving bartering. Bartering refers to trade where goods and services are exchanged directly, without the need for an intermediate currency. For example, two parties might trade two units of wool for three units of lumber.

Such trades do not have to be simple trades involving only two parties. Figure 12a depicts a situation in which more complex trades can occur: Party P_1 is willing to give two units of lumber, which P_2 wants. However, P_2 is only willing to give three units of ore, for which P_1 has no use. Instead, P_1 wants three units of wool in return for his lumber. Luckily, P_4 is willing to give three units of wool in return for three units of ore. This allows a cyclic trade: P_1 gives his lumber to P_2 , P_2 gives his ore to P_4 , and P_4 gives the wool to P_1 .

Of course, bartering is not restricted to lumber, ore, and wool. Other traded goods include books, rental cars, and production surpluses [49]. Centralized platforms play a crucial role in such modern-day bartering, as they provide two advantages over point-to-point bartering: They facilitate the discovery of bartering opportunities, and their holistic view on all bartering opportunities allows them to maximize the total number of trades⁷. However, centralized platforms also lead to privacy risks, as all parties have to disclose their offers and demands to the centralized platform. Additionally, these centralized platforms have the power to manipulate which parties get to trade, and which parties won't.

Motivated by these issues, Wüller [49] introduced a privacy-preserving bartering protocol. It allows a fixed number of parties to privately identify their trading partners such that the overall number of trades is maximized. Using SMPC techniques, it is ensured that each party only learns its own trading partners, without revealing information about the offers and demands to other parties.

Wüller's approach consists of five steps: First, the parties secretly input their offers, modeled by tuples (C_i^o, Q_i^o) stating that party P_i offers Q_i^o units of commodity C_i^o , and their demands (C_i^d, Q_i^d) that analogously encode that party P_i demands Q_i^d units of commodity C_i^d . Next, the protocol computes potential transfers of goods and constructs a so-called compatability graph G_C . As demonstrated by

the example in Figure 12b, this graph contains an edge (P_i, P_j) if and only if party P_i 's offer satisfies P_j 's demand, i. e., when $C_i^o = C_j^d$ and $Q_i^o \geq Q_j^d$. Through the use of SMPC techniques, the offers, demands, and the resulting compatibility graph remain secret.

The following step determines which trades should take place. This is the most complicated step of the protocol, and can be done using different sub-protocols which will be discussed in Appendices A.1 and A.2. All protocols result in a secret trade partner constellation, which is a subgraph of the compatability graph that only contains node-disjoint cycles, such as Figure 12c. That example trade partner constellation contains two trading cycles: (P_1, P_2, P_4) and (P_5, P_6) . In total, five parties get to perform a trade, which is the maximal possible number of trades in the presented example.

Once the trade partner constellation has been determined, each party learns its trading partners. Afterward, it is assumed that two trading parties are able to negotiate the actual trade quantity, since there is at least one acceptable quantity. Alternatively, protocols such as [52] can be used to fairly determine the trade quantity.

It should be noted that this protocol does not require all bartering parties to constantly be involved in the computation. Instead, there can be a different set of computing parties. The bartering parties secret-share their offers and demands among those computing parties, which perform the SMPC operations, and return the secret-shares of the results to the bartering parties. The set of bartering parties and computing parties are independent of each other, i. e., it is possible that the set of bartering parties equals the set of computing parties, but it is also possible that they do not overlap at all, or that just a subset of bartering parties are computing parties. This does not affect the security of the protocol as long as no large enough subset of computing parties collaborates⁸.

The description of our protocols also is not affected due to the ABB abstraction, but in this section "party" will refer to a bartering party, i. e., a party that provides inputs and receives outputs but that does not necessarily perform computations, unless explicitly stated otherwise.

In the following, this section will focus on the calculation of the trade-maximizing trade partner constellation, as this step profits from our shortest path protocol. Appendix A.1 will first describe prior protocols to find such trade partner constellations along with some novel optimizations. After, Appendix A.2 proposes a novel protocol to find the trade partner constellation that uses our shortest path protocol as sub-protocol. These protocols are then evaluated and compared in Appendix A.3.

A.1 Prior trade partner constellation protocols

Prior trade-maximizing trade partner constellation protocols can be divided into protocols that employ a brute-force approach to find the trade-maximizing trade partner constellation, and protocols that reduce the trade-maximization problem to a bipartite max-weight matching problem and use the Hungarian method to solve it. Those protocols will be discussed separately, starting with the former category.

⁷We will focus on systems that maximize the number of trades for simplicity. It is possible to adapt the following protocols to a limited set of other welfare functions.

⁸The number of computing parties that have to collaborate to break security guarantees depends on the used SMPC primitive.

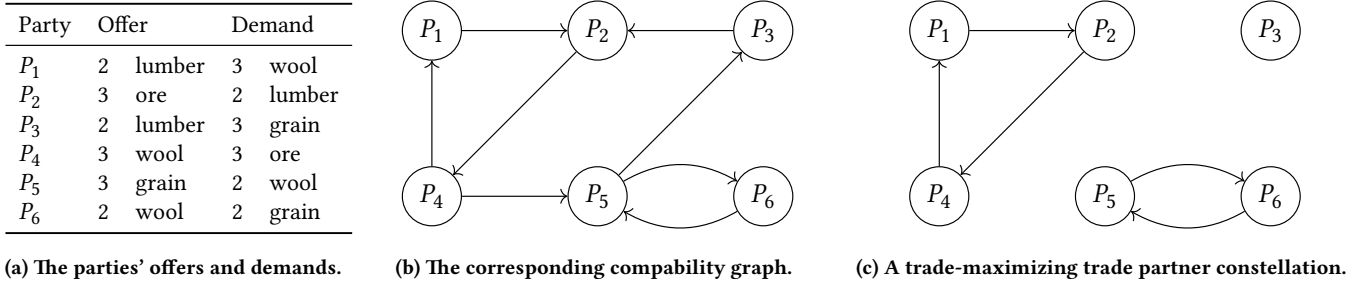


Figure 12: An example bartering situation.

A.1.1 Subgraph-Checking Protocols. Subgraph-checking protocols employ a brute-force approach, where all potential trade partner constellations are evaluated and the feasible trade-maximizing trade partner constellation is returned. More precisely, the parties first agree on a set of potential trade partner constellations G^{PTPC} . Commonly, this set contains all trade partner constellations over m parties, i. e., all graphs with m nodes that only contain node-disjoint cycles. Alternatively, those potential trade partner constellations are restricted to cycles of length two and three, justified by the higher failure probability of longer trade cycles as just one failing trade compromises the whole trading cycle.

Importantly, the set of potential trade partner constellations is selected a-priori, i. e., independently of the secret inputs, and therefore can be publicly known. But it also contains infeasible trade partner constellations, which need to be filtered out. This is done by checking which potential trade partner constellations are subgraphs of the compatibility graph. Wüller et al. [51] do so by multiplying the entries of the compatibility graph's adjacency matrix corresponding to the (publicly known) edges of potential trade partner constellations. When there are m bartering parties, this results in $\mathcal{O}(m \cdot |G^{\text{PTPC}}|)$ multiplications, all of which can be performed in $\mathcal{O}(\log m)$ rounds. The following ARGMAX then requires $\mathcal{O}(|G^{\text{PTPC}}|)$ comparisons over $\mathcal{O}(\log |G^{\text{PTPC}}|)$ rounds⁹. Overall, the initial protocol requires $\mathcal{O}(\log |G^{\text{PTPC}}|)$ rounds, and $\mathcal{O}(m \cdot |G^{\text{PTPC}}|)$ communication. This protocol was further optimized in [50] by avoiding redundant multiplications during the subgraph checks. Still, the resulting protocol is very slow [49].

We optimize the subgraph checking using a novel idea: Instead of multiplying the weight of infeasible trade partner constellations, we subtract a large penalty value. All infeasible trade partner constellations then have much lower weights than the feasible trade partner constellations, and thus will be ignored by the following ARGMAX¹⁰. This change eliminates all the communication in the subgraph checking phase of the protocol, as the penalty can be computed using local operations only. Hence, the only remaining communication occurs during the ARGMAX and the extraction of the trade partners, resulting in $\mathcal{O}(\log |G^{\text{PTPC}}|)$ rounds and $\mathcal{O}(|G^{\text{PTPC}}|)$ communication and computation.

A.1.2 The Hungarian method. A completely different approach is followed by [53]. This protocol reduces the trading-maximization

⁹Note that we applied [18] to parallelize the ARGMAX.

¹⁰To ensure that there is at least one feasible trade partner constellation, an "empty" constellation without any edges can be added to G^{PTPC} .

Protocol 9 Optimized subgraph checking. The modifications to [51] are highlighted in blue.

```

1: function FINDTRADEPARTNERCONSTELLATION( $[G_C], G^{\text{PTPC}}$ )
    $\triangleright G_{C,i,j} = 1 \Leftrightarrow \text{Edge}(i,j) \in G_C$ 
2:   for  $1 \leq i, j \leq m$  do  $m$  is the number of bartering parties.
3:      $[Penalty_{i,j}] \leftarrow (m+1) \cdot (1 - [G_{C,i,j}])$ 
4:   for  $1 \leq i \leq |G^{\text{PTPC}}|$  do
5:     Let  $G_i$  be the  $i$ 'th graph of  $G^{\text{PTPC}}$ .
6:     Further, let  $w$  be the number of edges in  $G_i$ .
7:      $[Weights_i] \leftarrow w - \sum_{\text{edge}(i,j) \in G_i} [Penalty_{i,j}]$ 
8:    $[c] \leftarrow \text{ARGMAX}([Weights])$ 
9:   Secretely determine and return the trade partners of  $G_c$ .
```

problem to a bipartite max-weight matching, and then uses the Hungarian method to solve the max-weight matching problem.

The reduction to the bipartite max-weight matching is demonstrated in Figure 13: For each node P_i in the compatibility graph (Figure 13a), a "giving node" P_i^g and a "receiving node" P_i^r are created. Then, each edge (P_i, P_j) in the compatibility graph is converted into an edge (P_i^g, P_j^r) of weight two, as shown by the solid edges in Figure 13b. Finally, an edge (P_i^g, P_i^r) of weight one is added for each party P_i , as shown by the dashed edges.

The core idea of this reduction is that if two nodes P_i^g and P_j^r are matched, then P_i gives their goods to P_j . In the special case that P_i^g and P_i^r are matched, P_i does not participate in any trade. The bipartite graph is constructed in a manner such that the max-weight matching always covers all nodes, i. e., all parties will either be assigned one giving and one receiving trading partner, or will be assigned to themselves. Hence, the max-weight matching neatly maps to a trade partner constellation. Further, it is ensured that the max-weight matching corresponds to a trade-maximizing trade partner constellation, as the (solid) trading edges have higher weights than the (dashed) non-trading edges.

In the context of SMPC protocols, the bipartite graph can easily be constructed using local computation only. However, finding the max-weight matching on that graph is more complicated. In [53], the Hungarian method is adapted into a privacy preserving protocol. On an intuitive level, this approach starts with a small set of "allowed edges", that only contains the highest-weight edges, and finds an unweighted matching using these allowed edges only. If

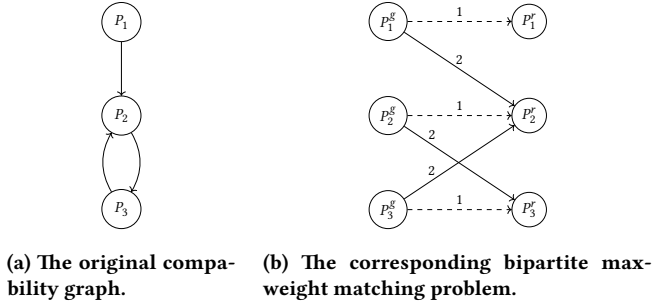


Figure 13: A visualization of the reduction from the trade-maximization problem to the bipartite max-weight matching problem.

this matching covers all nodes, a max-weight matching has been found. Otherwise, edges of lower weight are carefully added to the allowed edges, and a new unweighted matching is calculated. This is repeated until the unweighted matching covers all nodes.

The technical details of [53] will not be covered in this section, as they are not necessary to understand our changes. These are limited to switching the sub-protocol to find the unweighted matchings and the BFS sub-protocol to the current state-of-the-art [27]. With these changes, the protocol requires $\mathcal{O}(m^2 \log m)$ rounds and $\mathcal{O}(m^5 \log m)$ communication and computation to find the trade-maximizing trade partner constellation of m bartering parties.

It should be noted that Loruenser et al. [41] also have adapted the Hungarian method to a privacy-preserving protocol to solve a different problem. However, their protocol is not data-oblivious, and would leak at least the number of trades in the resulting trading partner constellation. Therefore, we have not further considered [41].

A.2 Our trade partner constellation protocol

Our protocol to find the trade-maximizing trade partner constellation is fairly similar to the Hungarian method approach, but it reduces to the min-cost max-flow problem instead. Figure 14 demonstrates this reduction for the same example as Figure 13: Again, a “giving node” P_i^g and a “receiving node” P_i^r are created for each bartering party P_i . The edges connecting these nodes also are very similar to the bipartite max-weight reduction: The dashed “non-trading” edges from P_i^g to P_i^r have capacity one and cost one, while the solid “trading” edges have capacity one and cost zero. However, the reduction has two additional nodes: A source node s , that is connected to all giving nodes by edges with capacity one and cost zero, and a sink node t that is connected to all receiving nodes by edges with capacity one and cost zero. The reduced problem then consists of finding a flow of minimal cost that sends as much flow as possible from the source node s to the sink node t .

The resulting flow will always send as many flow units from the source to the sink as there are bartering parties. Hence, each party will have an assigned trading partner, represented by an edge (P_i^g, P_j^r) with flow being sent along that edge. These edges are interpreted as party P_i giving their goods to party P_j . If a party P_i is assigned to itself, i. e., flow is sent along the edge (P_i^g, P_i^r) , then P_i does not participate in any trade. The min-cost flow will maximize

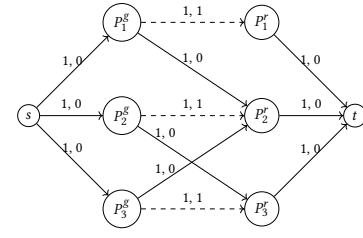


Figure 14: The min-cost max-flow graph corresponding to the compatibility graph of Figure 13a.

the use of trading edges, i. e., edges (P_i^g, P_j^r) with $i \neq j$, as these edges are cheaper than the non-trading edges. Hence, the min-cost max-flow in the reduced graph neatly maps to a trade-maximizing trade partner constellation.

Further the reduced min-cost max-flow problem can easily be constructed with local computations only. Solving the resulting problem is more complicated. We apply the successive shortest path approach, which finds the cheapest path from the source to the sink that still has capacity left, and sends as much flow as possible along that path [25]. This is repeated until no paths from the source to the sink have any residual capacity left, at which point a min-cost max-flow has been found [25].

Protocol 10 implements this approach. After constructing the min-cost max-flow problem in Lines 2 to 13, it initializes the flow. Next, it performs m augmenting iterations, where m is the number of bartering parties. Each iteration starts by constructing the adjacency matrix of the residual graph, which is then turned into a distance matrix by setting the length of edges to the corresponding costs, and adding ∞ to non-edges.

Next, the cheapest, i. e., shortest, path in the residual graph is found using our shortest path protocol. Protocol 10 hardcodes the successor calculation to the Floyd-Warshall sub-protocol, as the distance matrix might contain negative-length edges in later iterations, which would break the correctness of the Dijkstra sub-protocol. These negative-length edges could easily be avoided using local computations only by implementing the primal-dual technique presented in [25], which we did not do as our evaluation only reaches graph sizes where the Floyd-Warshall sub-protocol is faster than the Dijkstra sub-protocol anyways.

Once the cheapest path has been found, it is converted into an adjacency matrix Adj where $Adj_{i,j} = 1$ if the path uses the edge (i, j) . While it is possible to build such an adjacency matrix from only the step matrix in a single round and $\mathcal{O}(m^3)$ communication, Lines 21 to 24 reduce the communication to $\mathcal{O}(m^2)$ by observing that the successor matrix already is similar to the resulting adjacency matrix. Line 26 is then able to augment the flow directly, without determining the capacity of the path, as the graph construction only allows for augmenting paths with capacity one.

Since the graph structure ensures that total flow from the source to the sink is m , and each iteration increases the flow by one, the protocol will have found the min-cost max flow after m iterations.

Therefore, only the extraction of the trading partner remains using local computations only. Overall, the protocol requires $\mathcal{O}(m^2)$ rounds and $\mathcal{O}(m^4)$ communication and computation.

Protocol 10 Our trade partner constellation protocol.

```

1: function FINDTRADEPARTNERCONSTELLATION( $[G_C], G^{\text{PTPC}}$ )
     $\triangleright G_C$  is a  $m \times m$  matrix, and
     $\triangleright G_{Cij} = 1 \Leftrightarrow \text{Edge}(i, j) \in G_C$ 

    Build the min-cost max-flow problem:
2: for  $1 \leq i \leq m$  do
3:    $[Capacities_{1,i+1}] \leftarrow [1]$ 
4:    $[Capacities_{m+i+1,2m+1}] \leftarrow [1]$ 
5:    $[Capacities_{i+1,m+1+i}] \leftarrow [1]$ 
6:    $[Costs_{i+1,m+1+i}] \leftarrow [1]$ 
7:    $[Costs_{m+1+i,i+1}] \leftarrow [-1]$ 
8:   for  $1 \leq j \leq m, j \neq i$  do
9:      $[Capacities_{i+1,m+1+j}] \leftarrow [G_Cij]$ 
10:   $\overrightarrow{Source} \leftarrow E^{1,2m+2}$ 
11:   $\overrightarrow{Sink} \leftarrow E^{2m+1,2m+2}$ 
    Find the min-cost max-flow:
12:  for  $1 \leq i, j \leq 2m+2$  do
13:     $[Flow_{i,j}] \leftarrow [0]$ 
14:  for  $1 \leq iter \leq m$  do
    Build the residual distance matrix:
15:    for  $1 \leq i, j \leq 2m+2$  do
16:       $[ResidualAdj_{i,j}] \leftarrow [Capacities_{i,j}] - [Flow_{i,j}]$ 
17:       $[L_{i,j}] \leftarrow [Costs_{i,j}] + \infty \cdot (1 - [ResidualAdj_{i,j}])$ 
    Find the cheapest augmenting path:
18:     $[\overrightarrow{Succs}], [hasPath] \leftarrow \text{FLOYDWARSHALL}([L], \overrightarrow{Source}, \overrightarrow{Sink})$ 
19:     $[SuccMatr] \leftarrow \text{ToMATRIX}([\overrightarrow{Succs}], [hasPath])$ 
20:     $[Steps] \leftarrow \text{SuccMATRIXToPATH}([SuccMatr], \overrightarrow{Start})$ 
    Convert it to an adjacency matrix:
21:    for  $1 \leq i \leq 2m+2$  do in parallel
22:       $[visited] \leftarrow \sum_{j=1}^{2m+2} [Steps_{j,i}]$ 
23:      for  $1 \leq j \leq 2m+2$  do in parallel
24:         $[Adj_{i,j}] \leftarrow [visited] \cdot [SuccMatr_{i,j}]$ 
    Augment the flow:
25:    for  $1 \leq i, j \leq 2m+2$  do
26:       $[Flow_{i,j}] \leftarrow [Flow_{i,j}] + [Adj_{i,j}] - [Adj_{j,i}]$ 
    Extract the trading partners:
27:    for  $1 \leq i \leq m$  do
28:       $[Recipients_i] \leftarrow \sum_{j=1}^m j \cdot [Flow_{i+1,m+1+j}]$ 
29:  return  $[Recipients]$ 

```

A.3 Evaluation

The trade-maximizing trade partner constellation protocols were evaluated using the same setup as the main evaluation in Section 6¹¹, but only the LAN network setting was evaluated due to time constraints. The Hungarian method protocol and our successive shortest path protocol were evaluated with up to 50 bartering parties. The subgraph-checking protocol could only be evaluated up to ten

¹¹As in our main evaluation, all implementations, runtime logs, and plotting scripts are available in the artifact.

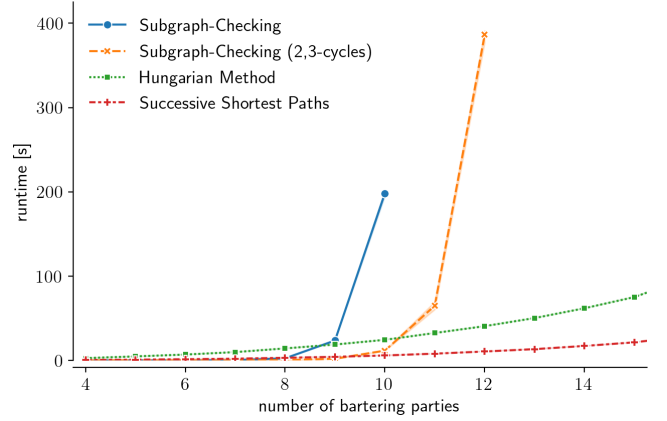


Figure 15: Runtimes of trade-maximizing trade partner constellation protocols with up to 15 bartering parties using the semi-honest primitive in the LAN setting.

parties with unrestricted cycle lengths, and 12 parties with cycle lengths restricted to two and three, as the machine’s 128GB of RAM is not enough for more bartering parties¹².

Nevertheless, the subgraph-checking protocols are very fast when there are less than nine bartering parties, as shown in Figure 15. They only require 6.9 seconds to find the trade partner constellation with unrestricted cycle lengths, and 1 second to find the trade partner constellation where cycle lengths are limited to two and three. In comparison, our successive shortest path protocol and the Hungarian method protocol, which both only support unrestricted cycle lengths, require 11.56, respectively 54 seconds.

However, the runtime of the subgraph-checking protocol grows exponentially, and our successive shortest path protocol outperforms the variant with unrestricted cycle lengths at nine bartering parties and beyond. The variant with cycles of length two and three is outperformed at ten bartering parties. This is explained by an exponential increase in the number of potential trade partner constellations. At six bartering parties, there are only 719 potential trade partner constellations with unrestricted cycle lengths. Hence, the subgraph-checking essentially is an ARGMAX over 719 values, which unsurprisingly is fast. However, at eight bartering parties, there are already 40 319 potential trade partner constellations and 3 628 799 at ten bartering parties. Thus, it is also not surprising that the subgraph-checking protocol quickly is outperformed by the other protocols whose runtimes scale polynomially.

Out of those two protocols, our successive shortest path protocol is, on average, 3.7 times faster than the optimized Hungarian method protocol as described in Appendix A.1.2. This speedup is explained by our protocol requiring both fewer rounds, e. g., 54 142 vs. 480 749 rounds at 50 bartering parties, and less communication, e. g., 237 gigabytes vs. 567 gigabytes at 50 bartering parties.

As shown by Figure 16, this speedup allows the overall bartering protocol to process more bartering parties within a fixed timeframe. For example, our successive shortest path protocol finds the trade-maximizing trade partner constellation of fifty bartering parties

¹²For example, each computing party would need at least 100 gigabytes of RAM for 13 bartering parties with cycles length restricted to two and three.

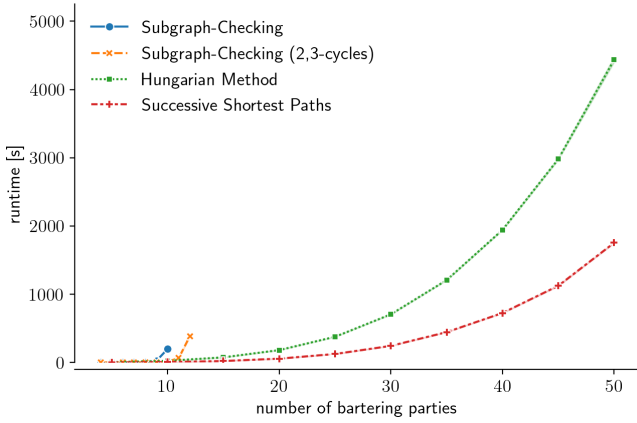


Figure 16: Runtimes of trade-maximizing trade partner constellation protocols using the semi-honest primitive in the LAN setting.

in just under 30 minutes. In the same timeframe, the fastest prior protocol is able to handle somewhere between 35 bartering parties (≈ 20 minutes) and 40 bartering parties (≈ 32 minutes). Hence, our protocol is able to handle at least 20% more bartering parties than prior protocols within 30 minutes.

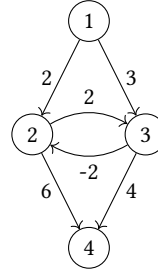
This relative improvement grows at smaller timeframes. For example, in roughly one minute (62 seconds), prior protocols can handle 14 bartering parties at most. Our protocol can compute the constellation for 20 parties in 56 seconds, an improvement of 42%. The only exceptions to this observation are very short timeframes below 5 seconds, where the subgraph-checking protocols outperform our successive shortest path protocol.

To summarize, we have presented a novel trade-maximizing trade partner constellation protocol that reduces to a min-cost max-flow problem, and uses the successive shortest path approach with our novel shortest path protocols to solve the reduced problem. When at least ten bartering parties are present, our new protocol is significantly faster than optimized versions of prior protocols, allowing privacy preserving bartering systems to handle between 20% and 42% more users.

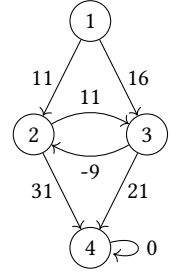
It should be noted that these improvements would not be possible without our shortest path protocol presented in Section 5, as our trade-maximization protocol needs the path from the source to the sink to augment the flow. Prior (length of) shortest path protocols could not have been used as they only compute distances.

B Shortest paths in $\mathcal{O}(\log^2 n)$ rounds

This section presents an alternative shortest path protocol that requires only $\mathcal{O}(\log^2 n)$ rounds, but is shown to be slow by Section 6 in practice. It is based on Protocol 11, a privacy-preserving adaptation of the min-addition algorithm [11] introduced by Anagreh et al. [5]. It repeatedly combines the distances of shortest paths with (up to) l steps into the distances of shortest paths with (up to) $2l$ steps. Hence, $\log_2(n-1)$ iterations are enough to calculate the distance with arbitrary number of steps, yielding the all-pair



(a) A graph containing a cycle of length zero.



(b) The transformed version of the example graph.

Figure 17: An example of the graph transformation.

shortest path distances in $\mathcal{O}(\log^2 n)$ rounds and $\mathcal{O}(n^3 \log n)$ communication.

Protocol 11 Min-addition length of shortest paths [5].

```

1: function MINADDDISTANCES( $[L]$ )
2:    $[D^0] \leftarrow [L]$ 
3:   for  $1 \leq \text{iter} \leq \lceil \log_2(n-1) \rceil$  do
4:     for  $1 \leq i, j \leq n$  do in parallel
5:        $[D_{i,j}^{\text{iter}}] \leftarrow \text{MIN}([D_{i,1}^{\text{iter}-1}] + [D_{1,j}^{\text{iter}-1}], \dots,$ 
         $[D_{i,n}^{\text{iter}-1}] + [D_{n,j}^{\text{iter}-1}])$ 
6:   return  $[D^{\lceil \log_2(n-1) \rceil}]$ 
    
```

As the other protocols by Anagreh et al. [5], this protocol only computes distances, and no paths. The techniques from Section 5 are not applicable, as the min-addition algorithm does not naturally produce predecessors. Instead, the node “in the middle of the path” can be found easily. Hence, Protocol 12 recovers a shortest walk from the start to the target node by recursively finding a shortest walk from the start to the middle node and from the middle node to the target node¹³. The middle node found by using the intermediary distances computed by Protocol 11, which preliminary tests revealed to be faster than explicitly storing middle nodes.

This protocol is not sufficient to find the shortest path, as it needs to be ensured that the graph contains a shortest walk with $2^{\lceil \log_2(n-1) \rceil}$ steps, and because it may contain cycles of length zero. We fix both of these issues by introducing the graph transformation defined by Definition B.1. As exemplified by Figure 17, this transformation adds a single-node loop of weight zero to the target node that ensures the existence of a $2^{\lceil \log_2(n-1) \rceil}$ -step walk in the first place. It also multiplies all edge distances by a large factor, and adds 1 afterward. This eliminates multi-node cycles with length zero, such as the cycle between nodes 2 and 3. As shown by Lemmas 1 and 2, the graph transformation forces the shortest walk to first match the shortest path until the target node is reached, at which point it repeatedly walks through the single-node loop.

Definition B.1. Let $G = (V, E, l)$ be a graph with edge lengths, and $t \in V$ the target node. The *transformed graph* is defined as

¹³The path produced by the protocol might contain cycles, and hence is called a walk.

Protocol 12 Reconstruction of the shortest walk.

```

1: function RECOVERWALK  $\left( \left[ \overrightarrow{Start} \right], \left[ \overrightarrow{End} \right], steps, \right.$ 
    $\left. \left[ D^0 \right], \dots, \left[ D^l \right], \right.$ 
    $\left. \left[ walkDistance \right], \right.$ 
    $\left. \left[ Result \right], offset \right)$ 

2:   if  $steps = 1$  then
3:      $\left[ Result_{offset,*} \right] \leftarrow \left[ \overrightarrow{Start} \right]$ 
4:      $\left[ Result_{offset+1,*} \right] \leftarrow \left[ \overrightarrow{End} \right]$ 
5:   return

   Find a middle node:
6:    $\left[ \overrightarrow{StartToAny} \right] \leftarrow \left[ \overrightarrow{Start} \right] \cdot \left[ D^{\lfloor \log_2(steps+1) \rfloor} \right]$ 
7:    $\left[ \overrightarrow{AnyToEnd} \right] \leftarrow \left[ D^{\lfloor \log_2(steps+1) \rfloor} \right] \cdot \left[ \overrightarrow{End} \right]$ 
8:    $\left[ \overrightarrow{Total} \right] \leftarrow \left[ \overrightarrow{StartToAny} \right] + \left[ \overrightarrow{AnyToEnd} \right]$ 
9:   for  $1 \leq k \leq n$  do in parallel
10:     $\left[ \overrightarrow{Mid}_k \right] \leftarrow \left[ \overrightarrow{Total}_k \right] \leq \left[ walkDistance \right]$ 
11:     $\left[ \overrightarrow{Mid}' \right] \leftarrow \text{UNITVECTOROFFIRSTONE} \left( \left[ \overrightarrow{Mid}' \right] \right)$ 
   Recover the subpaths:
12:    $\left[ firstHalfDistance \right] \leftarrow \sum_{i=1}^n \left[ \overrightarrow{StartToAny}_i \right] \cdot \left[ \overrightarrow{Mid}'_i \right]$ 
13:   in parallel do
14:     RECOVERWALK  $\left( \left[ \overrightarrow{Start} \right], \left[ \overrightarrow{Mid}' \right], \frac{steps}{2}, \left[ D^0 \right], \dots, \left[ D^l \right], \right.$ 
        $\left. \left[ firstHalfDistance \right], \left[ Result \right], offset \right)$ 

15:   and
16:     RECOVERWALK  $\left( \left[ \overrightarrow{Mid}' \right], \left[ \overrightarrow{End} \right], \frac{steps}{2}, \left[ D^0 \right], \dots, \left[ D^l \right], \right.$ 
        $\left. \left[ walkDistance \right] - \left[ firstHalfDistance \right], \right.$ 
        $\left. \left[ Result \right], offset + \frac{steps}{2} \right)$ 

```

$G_t^T = (V, E', l_t^T)$ with $E' = E \cup \{(t, t)\}$ and

$$l_t^T(i, j) = \begin{cases} (2^{\lfloor \log_2(n-1) \rfloor} + 1) \cdot l(i, j) + 1 & \text{if } i \neq j \\ 0 & \text{if } i = j = t \end{cases}$$

Definition B.2. Let $w = (v_1, \dots, v_k)$ be a walk, and $t \in V$ the target node. Further, let i be the smallest index such that $v_i = t$. Then, the loop-cut version of w is defined as $R_t(w) = (v_1, \dots, v_i)$.

LEMMA 1. Let $G = (V, E, l)$ be a graph with edge lengths and without cycles of negative length, and w be a shortest $2^{\lfloor \log_2(n-1) \rfloor}$ -step walk from s to t in G_t^T . Then $R_t(w)$ is a (cycle-free) path in G .

PROOF. This is proven by contradiction: If $R_t(w)$ is not cycle-free, then a $2^{\lfloor \log_2(n-1) \rfloor}$ -step walk w' from s to t can be built that is shorter than w , i.e., such that $l_t^T(w') < l_t^T(w)$. This contradicts the premise that w is a shortest walk.

Concretely, this alternative walk is built by removing a cycle from $R_t(w)$ and then padding it with single-node loops until it has $2^{\lfloor \log_2(n-1) \rfloor}$ steps again. To do so, let $R_t(w) = p_1 \parallel c \parallel p_2$ where c is a cycle. The alternative path is defined as $w' = p_1 \parallel p_2 \parallel (t, \dots, t)$.

To show that $l_t^T(w') < l_t^T(w)$, note that G does not contain cycles of negative length. It follows that

$$\begin{aligned}
 l(c) &= \sum_{k=1}^{|c|-1} l(c_k, c_{k+1}) \geq 0 \\
 \Rightarrow l_t^T(c) &= \sum_{k=1}^{|c|-1} (2^{\lfloor \log_2(n-1) \rfloor} + 1) \cdot l(c_k, c_{k+1}) + 1 \\
 &= (2^{\lfloor \log_2(n-1) \rfloor} + 1) l(c) + |c| - 1 \geq |c| - 1 > 0
 \end{aligned}$$

Further, since the single-node loop has length zero in G_t^T , it holds that $l_t^T(w') = l_t^T(w) - l_t^T(c)$, which is less than $l_t^T(w)$. Hence, w' is shorter than w , which concludes the proof. $\square$

LEMMA 2. Let $G = (V, E, l)$ be a graph with edge lengths and without cycles of negative length, and w be a shortest $2^{\lfloor \log_2(n-1) \rfloor}$ -step walk from s to t in G_t^T . Further, let p be a shortest path from s to t in G . Then $l(R_t(w)) = l(p)$, i.e., $R_t(w)$ is a shortest path.

PROOF. Again, this is proven by contradiction: Assume that $R_t(w)$ is not a shortest path from s to t in G . Then there must be a shorter path $p' = (v_1, \dots, v_k)$ from s to t in G , i.e., $v_1 = s$, $v_k = t$, and $l(p') < l(w)$. Analogously to Lemma 1, this path is turned into a $2^{\lfloor \log_2(n-1) \rfloor}$ -step walk w' by appending v_k until the number of steps has been reached. It remains to show that $l_t^T(w') < l_t^T(w)$:

$$\begin{aligned}
 l_t^T(w') &= \sum_{i=1}^{2^{\lfloor \log_2(n-1) \rfloor}} l_t^T(v_i, v_{i+1}) \\
 &= \sum_{i=1}^{k-1} (2^{\lfloor \log_2(n-1) \rfloor} + 1) l(v_i, v_{i+1}) + 1 \\
 &\leq (2^{\lfloor \log_2(n-1) \rfloor} + 1) l(p) + 2^{\lfloor \log_2(n-1) \rfloor} \\
 &< (2^{\lfloor \log_2(n-1) \rfloor} + 1) (l(p) + 1) \leq l_t^T(w)
 \end{aligned}$$

Hence, w is not a shortest $2^{\lfloor \log_2(n-1) \rfloor}$ -step walk from s to t in G_t^T , which contradicts the premise. $\square$

Hence, Protocol 13 can calculate the shortest path by first applying the graph transformation, which does not require any communication, running the min-addition distance calculation and walk recovery, and finally cutting of the single-node loops in constant rounds. In total, the protocol requires $\mathcal{O}(\log^2 n)$ rounds, and $\mathcal{O}(n^3 \log n)$ communication and computation. Finally, the protocol is data-oblivious and does not reveal any secret values. It therefore is trivially secure in the ABB model.

C Leakages in prior shortest-distance protocols

This section discusses a small and subtle leakage in (length of) shortest path protocols by Aly and Cleemput [2] and by Anagreh et al. [5]. Both protocols are inspired by Dijkstra's algorithm and follow the same approach presented in Section 3.1, which shuffles the nodes and reveals the shuffled index of the unvisited node closest to the start node. This enables efficient updates of the node's neighbors' distances. It is claimed that this approach is secure since the shuffling hides the real order the nodes are visited. Hence, this approach can be simulated by choosing a random permutation of

Protocol 13 The $\mathcal{O}(\log^2 n)$ round shortest path protocol.

```

1: function SHORTESTPATH( $[L], [\vec{Start}], [\vec{End}]$ )
    Determine the walk:
2:   for  $1 \leq i, j \leq n$  do
3:      $[D^0_{i,j}] \leftarrow \begin{cases} (2^{\lceil \log_2(n-1) \rceil} + 1) \cdot [L_{i,j}] + 1 & \text{if } i \neq j \\ (1 - [\vec{End}_i]) \cdot \infty & \text{if } i = j \end{cases}$ 
4:    $[D^1], \dots, [D^{\lceil \log_2(n-1) \rceil}] \leftarrow \text{MINADDDISTANCES}([D^0])$ 
5:    $[\text{pathLength}] \leftarrow [\vec{Start}] \cdot [D^{\lceil \log_2(n-1) \rceil}] \cdot [\vec{End}]^T$ 
6:   RECOVERWALK( $[\vec{Start}], [\vec{End}], \text{stepCount},$ 
        $[D^0], \dots, [D^{\lceil \log_2(n-1) \rceil}], [\text{pathLength}], [\text{Path}], 0$ )
    Remove the cycles:
7:    $[\text{isRealPath}] \leftarrow [\text{pathLength}] \leq \frac{(2^{\lceil \log_2(n-1) \rceil} + 1) \cdot \infty + 2}{2}$ 
8:   for  $1 \leq i \leq j$  do in parallel
9:      $[\text{isSameNode}] \leftarrow \sum_{j=1}^n [\text{Path}_{i-1,j}] \cdot [\text{Path}_{i,j}]$ 
10:     $[\text{keep}] \leftarrow [\text{isRealPath}] \cdot (1 - [\text{isSameNode}])$ 
11:    for  $1 \leq j \leq n$  do in parallel
12:       $[\text{Path}'_{i,j}] \leftarrow [\text{keep}] \cdot [\text{Path}_{i,j}]$ 
13:   return  $[\text{Path}']$ 

```

$1, \dots, n$ as revealed node order. However, [2, 5] both miss a small detail that thwarts this argument: In both cases, the protocols use the blinding permutation as tie-breaker when multiple nodes have the same distance to the starting node.

Consider, for example, Protocol 14, the ARGMIN sub-protocol used by Aly and Cleemput [2]. It performs a straightforward linear search to find the node with the minimal distance to the start node. However, if there are multiple nodes that all have the equal (minimal) distance to the start node, it will just return the first of these nodes it encounters, which is the node with the lowest shuffled index. This is also the case for the ARGMIN sub-protocol used by Anagreh et al. [5], which splits the input vector in two halves and recursively finds the minimum and the corresponding index in both halves. If the minima of both halves are equal, it always returns the lower shuffled index.

Protocol 14 ARGMIN sub-protocol used by Aly and Cleemput [2].

```

1: function ARGMIN( $[\vec{DistToStart}], i$ )
2:    $\triangleright i$  is used to reduce the search space in the  $i$ 'th Dijkstra
3:    $\triangleright$  iteration. This is not relevant for the leakage.
4:    $[d] \leftarrow [\infty]$ 
5:    $[v] \leftarrow [\perp]$ 
6:   for  $i \leq j \leq n$  do
7:      $[\text{cond}] \leftarrow [\vec{DistToStart}_j] < [d]$ 
8:      $[v] \leftarrow [\text{cond}] ? j : [v]$ 
9:      $[d] \leftarrow [\text{cond}] ? [\vec{DistToStart}_j] : [d]$ 
10:  return  $[v]$ 

```

This behavior becomes problematic when multiple nodes have the same distance to the start node. In that case, the gates will always

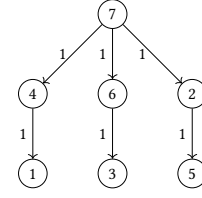


Figure 18: An example graph on which the leakage occurs. The node labels are the shuffled node indices.

select the node with the lowest shuffled index. In the following iteration, the node with the second-lowest shuffled index will be selected, and so on. As a result, the revealed shuffled node indices are strictly increasing as long as the corresponding nodes have the same distance to the starting node.

Consider, for example, the graph presented in Figure 18. No matter which permutation is used during the protocol executions, the second revealed node index will always be smaller than the third revealed node index. As a consequence, some traces will never occur, such as “7 6 4 2 1 3 5”. This breaks the security arguments by Aly and Cleemput [2] and Anagreh et al. [5] that claim that the revealed trace is a uniformly random permutation of $1, \dots, n$.

Next to the academic discussion of the leakage, we want to provide some intuition on the leakage: Each time that the revealed shuffled node index decreases, it is revealed that the distance to the starting node increases. On the other hand, it is possible for the distance to the starting node to increase without a decrease in the shuffled node indices. This allows deriving further information about the graph, e. g., a lower bound on the layers in the graph, and an upper bound for the size of each layer.

Depending on the application of the protocols, the practical implications of this leakage varies significantly. If it is known a-priori from the use-case that each node has a distinct distance to the start node, this leakage becomes irrelevant. On the other hand, when these protocols are used as part of larger protocols, e. g., to find min-cost-max-flows as in Appendix A, the leakage might reveal information about the state of the larger protocol, which ultimately could reveal more sensitive information.

Luckily, the leakage can be avoided by making the node choice independent of the shuffling permutation. To do so, we propose the original, unshuffled, node index as tie-breaker for the case that two nodes have the same distance to the starting node. As shown in Protocol 15, this is done by first shuffling a vector containing the values $1, \dots, n$. Afterward, the tie-breaker is incorporated into the node selection by first multiplying all distances to the start node by n , and then adding the tie-breaker in Line 8. This allows selecting the nearest node with any ARGMIN gate, i. e., this fix is compatible with the protocol by Aly and Cleemput [2] as well as the protocol by Anagreh et al. [5]. It is assumed that the ARGMIN procedure ignores already visited nodes.

The performance penalty of this fix should be minimal. This is because the only additional communication introduced by the fix is the shuffling of the tie-breaker vector, which can be done in parallel to the shuffling of the distance matrix and the distance vector. Building the temporary distance vector during the iterations

of the protocols is done using local computations only, i. e., no communication overhead occurs during the iterations compared to the leaky Dijkstra protocols.

Protocol 15 Dijkstra protocol without leakage.

```

1: function DIJKSTRA ( $[L], [\overrightarrow{Start}]$ )
2:   for  $1 \leq i \leq n$  do
3:      $[\overrightarrow{Dist}_i] \leftarrow (1 - [\overrightarrow{Start}_i]) \cdot \infty$ 
4:      $[\overrightarrow{TieBreaker}_i] \leftarrow [i]$ 
5:   Permute  $[L], [\overrightarrow{DistToStart}], [\overrightarrow{TieBreaker}]$ 
6:   for  $1 \leq \text{iter} \leq n$  do
7:     for  $1 \leq j \leq n$  do
8:        $[\overrightarrow{Tmp}_j] \leftarrow (n + 1) \cdot [\overrightarrow{Dist}_j] + [\overrightarrow{TieBreaker}_j]$ 
9:      $v \leftarrow \text{REVEAL}(\text{ARGMINOFUNVISITED}([\overrightarrow{Tmp}]))$ 
10:    Update the distances of  $v$ 's neighbors and
        mark it as visited.
11:    Unpermute and return  $[\overrightarrow{DistToStart}]$ .
```

Further, the protocol stays correct due to the multiplication of the distances by $n + 1$, as the tie-breaker can add at most n to the temporary distances. Finally, we show that this fix actually fixes the leakage. This requires a brief clarification of the math around shuffled vectors, which tends to be counterintuitive. For example, when $\vec{B}$ is the shuffled version of $\vec{A}$ then $\vec{B}_i = \vec{A}_{\sigma^{-1}(i)}$, as shown in Equation (1).

$$\vec{B} = \sigma(\vec{A}) \Leftrightarrow \forall i : \vec{B}_{\sigma(i)} = \vec{A}_i \Leftrightarrow \forall i : \vec{B}_i = \vec{A}_{\sigma^{-1}(i)} \quad (1)$$

It gets more complicated when an ARGMIN over a shuffled vector is calculated. Equation (2) shows that an ARGMIN over a shuffled vector or function is equal to the inverse permutation of the ARGMIN on the unshuffled vector or function. The equation uses the function notation with the constraint that f is injective, i. e., that each $f(j)$ is distinct. This ensures that the ARGMIN is well-defined without the need to specify a tie-breaker.

$$\begin{aligned}
& \sigma^{-1}(\text{ARGMIN}_{1 \leq j \leq n}(f(j))) \\
&= \sigma^{-1}(j \text{ such that } f(j) = \text{MIN}_k(f(k))) \\
&= \sigma^{-1}(j \text{ such that } i = \sigma^{-1}(j) \wedge f(\sigma(i)) = \text{MIN}_k(f(k))) \quad (2) \\
&= i \text{ such that } f(\sigma(i)) = \text{MIN}_k(f(k)) \\
&= \text{ARGMIN}_{1 \leq i \leq n}(f(\sigma(i)))
\end{aligned}$$

Using these tools, Theorem C.1 shows that the revealed traces of Protocol 15 indeed are simulatable by choosing a random permutation of $1, \dots, n$.

THEOREM C.1. *Protocol 15 is secure in the ABB model, assuming Lines 5, 10 and 11 of Protocol 15 and ARGMIN are implemented securely in the ABB model.*

PROOF. As stated in [4], it only needs to be shown that the revealed order of visited nodes can be simulated.

To do so, let $G = (V = \{1, \dots, n\}, L)$ be a graph with edge lengths and $v \in V$ be a starting node. Further, let $u^1, \dots, u^n$ be the trace of revealed node indices obtained from executing the Protocol 15 on

G, v without shuffling the nodes, and $\overrightarrow{DistToStart}^{\text{iter}}$ the state of the $\overrightarrow{DistToStart}$ vector before the ARGMIN is calculated in iteration iter.

We claim that this unshuffled trace is the same underlying trace for all executions of Protocol 15. In other words, if the protocol shuffles the nodes using the permutation σ and reveals the trace of shuffled node indices $t^1, \dots, t^n$, then $\sigma^{-1}(t^{\text{iter}}) = u^{\text{iter}}$.

This shown by induction over the iterations. Let $\overrightarrow{DistToStart}^{\text{iter}}$ be the distances before the ARGMIN is calculated in the iteration iter of Protocol 15 while using σ to shuffle the nodes. $\overrightarrow{DistToStart}^1 = \sigma(\overrightarrow{DistToStart}^1)$ holds by definition. Equation (3) then shows that $\sigma^{-1}(t^1) = u^1$, i. e., that the same node is selected in both the shuffled and the unshuffled protocol execution. Hence, the distance updates will behave analogously and $\overrightarrow{DistToStart}^2 = \sigma(\overrightarrow{DistToStart}^2)$.

It follows from induction that $\overrightarrow{DistToStart}^{\text{iter}} = \sigma(\overrightarrow{DistToStart}^{\text{iter}})$ and $\sigma^{-1}(t^{\text{iter}}) = u^{\text{iter}}$ for all iterations $1 \leq \text{iter} \leq n$.

$$\begin{aligned}
& \sigma^{-1}(t^{\text{iter}}) = \sigma^{-1}(\text{ARGMIN}(\overrightarrow{Tmp}^{\text{iter}})) \\
&= \sigma^{-1}(\text{ARGMIN}_{1 \leq j \leq n}((n + 1) \cdot \overrightarrow{DistToStart}^{\text{iter}}_j + \overrightarrow{TieBreaker}_j)) \\
&= \sigma^{-1}(\text{ARGMIN}_{1 \leq j \leq n}((n + 1) \cdot \overrightarrow{DistToStart}^{\text{iter}}_{\sigma(j)} + \sigma^{-1}(j))) \\
&= \text{ARGMIN}_{1 \leq i \leq n}((n + 1) \cdot \overrightarrow{DistToStart}^{\text{iter}}_{\sigma(i)} + i) \\
&= \text{ARGMIN}_{1 \leq i \leq n}((n + 1) \cdot \overrightarrow{DistToStart}^{\text{iter}}_i + i) = u^{\text{iter}} \quad (3)
\end{aligned}$$

In other words, all protocol executions on G, v have the same underlying unshuffled trace $u^1, \dots, u^n$. It remains to show that all shuffled traces are equally likely. As the unshuffled trace is independent of the used permutation, this is rather simple: The probability that any given shuffled trace $t^1, \dots, t^n$ is revealed equals the probability that the permutation σ maps $u^1, \dots, u^n$ to $t^1, \dots, t^n$, which is $\frac{1}{n!}$. Note that this argument could not be made if the unshuffled trace depends on the used permutation.

It follows that the revealed trace can be simulated by choosing a uniformly random permutation of $1, \dots, n$. Hence, Protocol 15 is secure in the ABB model. $\square$

D Additional proofs

In this section, statements from the main part are proven formally. Concretely, Appendix D.1 first shows that the trick employed in Section 5.1 to calculate successors instead of predecessors indeed results in successors. Appendix D.2 then shows that the concrete protocols presented in Section 5.1 produce cycle-free successors. Finally, Appendix D.3 proves that the shuffle-reveal path extraction sub-protocol is secure, i. e., that revealing shuffled node indices does not leak any information.

D.1 Correctness of successor computation

To show that the trick used to calculate successors instead of predecessors in Section 5.1 indeed results in the successors of a shortest path, Lemma 3 first notes that every shortest path in the transformed graph corresponds to a shortest path in the original graph.

LEMMA 3. Let $G = (V, E, l)$ be a graph with edge lengths, and $G' = (V, \{(w, v) \mid (v, w) \in E\}, l')$ with $l'(v, w) = l(w, v)$ be the corresponding transformed graph. Further let, $p' = (v_1, \dots, v_k)$ be a shortest path from $t \in V$ to $s \in V$ in G' . Then the $p = (v_k, \dots, v_1)$ is a shortest path from s to t in G .

PROOF. First, we note that $l(p') = l(p)$. Next, we assume p is not a shortest path from s to t in G , i. e., that there is an alternative path $q = (w_1, \dots, w_k)$ in G with $l(q) < l(p)$. Then we note that there is a corresponding path $q' = (w_k, \dots, w_1)$ from t to s in G' such that $l'(q') < l'(p')$. This contradicts the premise that p' is a shortest path. Hence, p is a shortest path too. $\square$

Note that the corresponding shortest path p in G to every shortest path p' in G' is just p' in reversed order. Hence, the predecessor of a node in p' is equal to the successor of the same node in p .

D.2 The successors are cycle-free

In the following, we prove that the successors produced by Protocol 2 and 3 do not contain cycles.

LEMMA 4. Let $G = (V, E, l)$ be a graph with edge lengths and without edges of negative length, $s \in V$, and Succs the successors produced by running the Dijkstra protocol on G, s . Then, for all reachable $t \in V \setminus \{s\}$, the path from s to t induced by the successors does not contain any cycles.

PROOF. Assume that the successors induce a cycle $(c_1, \dots, c_k, c_1)$. It follows that $\overrightarrow{\text{Dist}}_{c_k} = \overrightarrow{\text{Dist}}_{c_1} + \sum_{i=1}^{k-1} l(c_i, c_{i+1}) \geq \overrightarrow{\text{Dist}}_{c_1}$ since all edges have a non-negative length. However, c_k is only set to be the successors of c_1 if $\overrightarrow{\text{Dist}}_{c_k} + l(c_k, c_1) < \overrightarrow{\text{Dist}}_{c_1}$. As $l(c_k, c_1)$ is non-negative, this contradicts $\overrightarrow{\text{Dist}}_{c_k} \geq \overrightarrow{\text{Dist}}_{c_1}$. Hence, the successors cannot induce a cycle. $\square$

LEMMA 5. Let $G = (V, E, l)$ be a graph with edge lengths and without cycles of negative length, and Succs the successors produced by running the Floyd-Warshall protocol on G . Then, for all $i, j \in V, i \neq j$ such that j is reachable from i , the path from i to j induced by the successors does not contain any cycles.

PROOF. This is shown by induction over k , i. e., over the protocol's iterations. To start, we note that after the first iteration, i. e., for $k = 1$, the lemma holds since each path produced by Succs has at most one step and therefore cannot contain a cycle.

Next, assume that the lemma holds after k iterations of the protocol. By contradiction, we show that lemma also must hold after $k+1$ iterations: Assume that there are i, j such that $\text{Dist}_{i,k+1} + \text{Dist}_{k+1,j} < \text{Dist}_{i,j}$, but such that the corresponding path $p = p_{i,k+1} \parallel p_{k+1,j}$ does contain a cycle. This path then can be decomposed into the part prior to the cycle p'_1 , the cycle c , and the part after the cycle p'_2 , and the cycle-free version of the path can be constructed as $p' = p'_1 \parallel p'_2$.

Since the paths using nodes $\{1, \dots, k\}$ are cycle-free by the induction assumption, the cycle must contain the node $k+1$. Further, the node $k+1$ can appear only once in the complete path. Therefore p' only visits nodes in $\{1, \dots, k\}$. It follows that $l(p') \geq \text{Dist}_{i,j}$.

Finally, $l(c) \geq 0$ since G does not contain cycles of negative length. Therefore, $l(p) = l(p'_1) + l(c) + l(p'_2) \geq l(p') \geq \text{Dist}_{i,j}$,

violating the premise $\text{Dist}_{i,k+1} + \text{Dist}_{k+1,j} < \text{Dist}_{i,j}$. It follows that the lemma also must hold after $k+1$ iterations, and therefore also holds in general. $\square$

D.3 The shuffle-and-reveal path extraction is secure

In Section 5.2.2, we present Protocol 8, which extracts a path from the successor matrix by first adding fake nodes, shuffling all nodes, and then revealing the shuffled indices of the visited nodes. We argue that due to the shuffling of the nodes, the revealed shuffled node indices do not leak information, which we prove formally in Lemma 6.

LEMMA 6. Protocol 8 is secure in the ABB model, assuming the inputs are well-formed, i. e., each row of the successor matrix is either a unit-vector or only contains zeroes, $\text{Succs}_{i,i} = 0$ for all $1 \leq i \leq n$, and that $\overrightarrow{\text{Start}}$ is a unit-vector.

PROOF. First, we note that the control flow of Protocol 8 only depends on the publicly known number of nodes. Hence, its simulation is trivial, and we only need to show that the reveal (shuffled) node indices can be simulated.

We claim that this can be done by uniformly sampling n distinct values from $\{1, \dots, 2n-1\}$. Note that the probability of the simulator choosing any trace uniformly is $\frac{(n-2)!}{(2n-1)!}$.

Next, let the input to the protocol of the protocol be well-formed and arbitrary but fixed, and σ be the permutation used during a protocol execution. Further, let $t^1, \dots, t^n$ be the trace of shuffled node indices revealed by the protocol execution, and $u^{\text{iter}} = \sigma^{-1}(t^{\text{iter}})$ be the corresponding trace of unshuffled node indices. Finally, let $p = (v_1, \dots, v_k)$ be the path produced by the protocol on the inputs.

We note that for every possible σ the unshuffled trace $u^1, \dots, u^n$ always is $v_1, \dots, v_k, n+1, \dots, 2n-k-1$, i. e., it always starts with the resulting path followed by the chain of the fake nodes, independently of the used permutation.

This means that for this arbitrary but fixed well-formed input, the probability of any trace $t^1, \dots, t^n$ being revealed equals the probability of the permutation mapping $v_1, \dots, v_k, n+1, \dots, 2n-k-1$ to $t^1, \dots, t^n$. As the protocol chooses the permutation uniformly at random, this probability is $\frac{(n-2)!}{(2n-1)!}$.

Hence, the distribution of simulated traces equals the distribution of the real traces created by the protocol, and the simulated traces are not distinguishable from real protocol traces. $\square$